

剑道试炼

168道武侠风算法题 · 15章 · C++ + Python

第1章 持剑叩门

变量与输入输出

李少白第一次来到剑道宗山门前。梁嘉峰递给他两枚铁令："学会输入输出，才能踏入剑道的第一步。"

本章将学习变量、数据类型和输入输出——这是所有程序的基础。掌握如何读入数据、计算、输出结果，你就能写出人生中第一个程序了。

JD001: 铁令求和

简单 AcWing 1 | JD001

李少白第一次来到剑道宗山门前。梁嘉峰递给他两枚铁令，上面各刻着一个数。「加起来，报给我。」

输入格式

一行，两个整数A和B，用空格隔开。

输出格式

一个整数，即A+B的结果。

输入样例

3 4

输出样例

7

解题思路：

读入两个整数，输出它们的和。

复杂度分析：

时间复杂度：O(1) — 固定次数的输入输出操作

空间复杂度：O(1)

▶ 参考代码

JD002：铁令相乘

简单 AcWing 605 | JD002

梁嘉峰从腰间解下两枚铁令，随手抛给李少白。「上次是加法，」他说，「这次换个运算。」李少白接住一看，每枚铁令背面各刻着一个数字。赵晴儿在旁边提醒道：「还记得加法的三步吗？输入、处理、输出。只不过这次，处理的那一步从加号变成了乘号。」梁嘉峰点了点头：「乘法。报结果的时候，前面加上 PROD = 。」

输入格式

一行，两个整数A和B，用空格隔开。

输出格式

输出 `PROD =` 后跟 $A \times B$ 的结果。

输入样例

```
3 9
```

输出样例

```
PROD = 27
```

解题思路：

读入两个整数，输出它们的乘积。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的输入输出操作

空间复杂度： $O(1)$

► 参考代码

JD003：四令求差

简单 AcWing 608 | JD003

练功房桌上摆着四枚铁令。梁嘉峰说：「A和B相乘，C和D相乘，把两个积的差报给我。」

输入格式

一行，四个整数A、B、C、D，用空格隔开。

输出格式

输出 `DIFFERENCE =` 后跟 $A \times B - C \times D$ 的结果。

输入样例

```
5 6 7 8
```

输出样例

```
DIFFERENCE = -26
```

解题思路：

先乘后减： $A \times B - C \times D$ 。注意运算顺序和输出格式。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的输入输出操作

空间复杂度： $O(1)$

► 参考代码

JD004：集市算账

简单 AcWing 611 | JD004

宗门集市的账房里，算盘珠子噼啪作响。账房先生递来一张货单：「赵姑娘，帮我算算这笔账。」单上写着商品编号、购买数量和单价。赵晴儿看了一眼，把货单递给身旁的李少白：「你来。记住，商品编号只是标记，真正的计算只有数量乘以单价——分清哪些数据有用、哪些是干扰，这也是编程的基本功。」

输入格式

一行，三个数：商品编号（整数）、购买数量（整数）、单价（浮点数）。

输出格式

输出 `TOTAL =` 后跟总价（数量 $\times$ 单价），保留两位小数。

输入样例

```
12 1 5.30
```

输出样例

```
TOTAL = 5.30
```

解题思路：

总价 = 数量 $\times$ 单价。注意输入是一行三个数，商品编号只用于读入不需要参与计算。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的算术运算

空间复杂度： $O(1)$

► 参考代码

JD005：比武台

简单 AcWing 604 | JD005

山门内是一片圆形的比武台，青石铺地，平整如镜。李少白绕着边沿走了一圈，估出半径大约是 r 步。赵晴儿站在台中央，裙角被风吹起：「这块比武台有多大？面积 $=\pi \times r^2$ ， π 取3.14159。」她蹲下来用树枝在泥地上写了公式，「浮点数运算——小数的精度很重要，差之毫厘，谬以千里。」

输入格式

一个浮点数 r ，表示半径。

输出格式

输出 `A=` 后跟面积，保留4位小数。

输入样例

```
2.00
```

输出样例

```
A=12.5664
```

解题思路：

面积 $=\pi \times r^2$ ，保留4位小数。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的算术运算

空间复杂度： $O(1)$

► 参考代码

JD006：剑术考核

简单 AcWing 606 | JD006

入门考核的成绩单发下来了。李少白凑过去一看：剑术8.2分，心法6.5分。他正想取平均数，赵晴儿拦住了他：「等等——剑术占3.5成，心法占7.5成，权重不同，不能简单相加除以二。」梁嘉峰走过来，把权重写在石板上：「加权平均， $(\text{成绩A} \times \text{权重}3.5 + \text{成绩B} \times \text{权重}7.5) \div \text{总权重}11$ 。这比简单平均更能反映真实水平。」

本题输出前缀 `Average =` 为原题格式要求。

class="io-section">

输入格式

两行，每行一个浮点数，分别表示剑术成绩A和心法成绩B。

输出格式

输出 `Average =` 后跟加权平均分，保留5位小数。

输入样例

9.7
5.3

输出样例

Average = 6.70000

解题思路：

加权平均 = $(A \times 3.5 + B \times 7.5) / 11$ 。保留5位小数。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的算术运算

空间复杂度： $O(1)$

► [参考代码](#)

JD007：月底发饷

简单 AcWing 609 | JD007

月底了，账房里排起长队。李少白被叫去帮忙——一名弟子递来工牌，上面刻着工号。账房翻开考勤簿，念出两个数字：出工天数，和每日工钱。「算算实发多少，」账房说，「格式照规矩来：先打工号，再打 $SALARY = U\$$ ，保留两位小数。」赵晴儿在一旁轻声说：「这是多个输入、多行输出的练习——程序不只能算数，还能按规矩排版。」

输入格式

第一行一个整数，表示工号。第二行两个数：出工天数（整数）和每日工钱（浮点数）。

输出格式

第一行输出工号。第二行输出 $SALARY = U\$$ 后跟实发金额，保留两位小数。

输入样例

```
25
100 5.50
```

输出样例

```
NUMBER = 25
SALARY = U$ 550.00
```

解题思路：

实发金额 = 出工天数 × 每日工钱，保留两位小数。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的算术运算

空间复杂度： $O(1)$

▶ 参考代码

JD008：马匹脚力

简单 AcWing 615 | JD008

李少白骑马送信，跑了S公里，马匹消耗了L升草料汁。他想知道每升草料汁能支撑跑多少公里。

输入格式

一行，两个浮点数，分别表示路程（km）和草料消耗（L）。

输出格式

输出每升草料汁能跑的公里数，保留3位小数，后跟 `km/l`。

输入样例

```
500 35.0
```

输出样例

```
14.286 km/l
```

解题思路：

每升里程 = S / L ，保留3位小数，后跟" km/l"。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的算术运算

空间复杂度： $O(1)$

► 参考代码

JD009：铁球体积

简单 AcWing 612 | JD009

兵器库角落里，一颗实心铁球蒙着灰尘。李少白擦了擦，用绳子量出半径 r 。「这铁球有多重？」他自言自语。赵晴儿走过来：「先算体积——球的体积公式是 $V = (4/3)\pi r^3$ ， π 取3.14159。」她顿了顿，「注意4/3在代码里要写成4.0/3.0，不然整数除法会把小数部分丢掉，算出来的结果就差远了。」

输入格式

一个浮点数 r ，表示铁球的半径。

输出格式

输出 `VOLUME =` 后跟体积，保留3位小数。

输入样例

3

输出样例

VOLUME = 113.097

解题思路：

体积 = $4.0/3.0 \times 3.14159 \times r^3$ ，保留3位小数。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的算术运算

空间复杂度： $O(1)$

► 参考代码

JD010：三令比大

简单 AcWing 614 | JD010

练功房的白墙上，梁嘉峰用炭笔歪歪扭扭地写了三个数。他退后一步，抱着胳膊看李少白：

「三个数里挑最大的。」李少白想了想：「用 if 比较？」赵晴儿从书架后探出头：「也可以用 max 函数。但梁师兄想让你先理解比较的过程——把第一个数暂存起来，后面每遇到更大的就替换。这个思路，以后处理一万个数也是一样的。」

输入格式

一行，三个整数A、B、C，用空格隔开。

输出格式

输出 `Max =` 后跟三个数中的最大值。

输入样例

```
7 14 106
```

输出样例

```
Max = 106
```

解题思路：

用条件判断或max函数找最大值。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的算术运算

空间复杂度： $O(1)$

► 参考代码

JD011: 哨塔测距

简单 AcWing 616 | JD011

赵晴儿在沙盘上标了两个哨塔的坐标 (x_1, y_1) 和 (x_2, y_2) 。她说：「算算它们之间的直线距离。」

输入格式

一行，四个浮点数 x_1, y_1, x_2, y_2 ，用空格隔开。

输出格式

输出两点间的距离，保留两位小数。

输入样例

```
1.0 7.0 5.0 9.0
```

输出样例

```
4.4721
```

解题思路：

距离公式 $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ ，保留4位小数。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的算术运算

空间复杂度： $O(1)$

► 参考代码

JD012: 圆盘开孔

简单 AcWing 613 | JD012

赵晴儿在沙盘上画了五个图形，每个图形都用A、B、C三个数来定义。她让李少白算出每个图形的面积：- 三角形：底A高C，面积 = $A \times C \div 2$ - 圆形：半径C，面积 = $\pi \times C^2$ - 梯形：上底A下底B高C，面积 = $(A+B) \times C \div 2$ - 正方形：边长B，面积 = B^2 - 长方形：长A宽B，面积 = $A \times B$
 π 取3.14159。

输入格式

一行，三个浮点数R、r1、r2。

输出格式

五行，分别输出 TRIANGULO（三角形）、CIRCULO（圆形）、TRAPEZIO（梯形）、QUADRADO（正方形）、RETANGULO（长方形）的面积，各保留3位小数。格式为 **名称：面积**。

输入样例

```
3.0 4.0 5.2
```

输出样例

```
TRIANGULO: 7.800
CIRCULO: 84.949
TRAPEZIO: 18.200
QUADRADO: 16.000
RETANGULO: 12.000
```

解题思路：

三角形 = $A \times C / 2$ ，圆形 = $\pi \times C^2$ ，梯形 = $(A+B) \times C / 2$ ，各保留3位小数。

复杂度分析:

时间复杂度: $O(1)$ — 固定次数的算术运算

空间复杂度: $O(1)$

► [参考代码](#)

JD013：水钟报时

简单 AcWing 654 | JD013

宗门的水钟记录的是总秒数，但掌门要求用「时:分:秒」的格式汇报。赵晴儿让李少白帮忙转换。

输入格式

一个整数N，表示总秒数。

输出格式

输出 `H:M:S` 格式的时间。

输入样例

556

输出样例

0:9:16

解题思路：

时=秒 $\div$ 3600，分=余数 $\div$ 60，秒=最终余数。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的算术运算

空间复杂度： $O(1)$

► 参考代码

JD014：钱庄换银

简单 AcWing 653 | JD014

李少白去镇上钱庄换碎银。掌柜说：「你要换多少文？我用100、50、20、10、5、2、1面额的铜钱给你。」需要知道每种面额各要几枚。

本题输出格式中的 `nota(s) de R$` 为巴西货币单位，属于原题格式要求。

class="io-section">

输入格式

一个整数N，表示要换的文数。

输出格式

第一行输出总金额N。接下来7行，按面额从大到小，每行输出 `X nota(s) de R$ Y,00`，其中X是张数，Y是面额。

输入样例

```
576
```

输出样例

```
576
5 nota(s) de R$ 100,00
1 nota(s) de R$ 50,00
1 nota(s) de R$ 20,00
0 nota(s) de R$ 10,00
1 nota(s) de R$ 5,00
0 nota(s) de R$ 2,00
1 nota(s) de R$ 1,00
```

解题思路：

贪心：从大到小依次整除。先输出总金额N，再输出每种面额的张数。

复杂度分析:

时间复杂度: $O(n)$ — 贪心遍历每种面额, 面额数为常数

空间复杂度: $O(1)$

► [参考代码](#)

第2章 歧路逢生

条件判断与分支

山道分岔，石碑指路。赵晴儿说："条件判断让程序学会了选择——根据不同的条件，走不同的路。"

本章学习 if/else 条件判断——让程序根据不同的输入做出不同的反应。这是编程从"计算"走向"决策"的关键一步。

JD015：石碑之谜

简单 AcWing 665 | JD015

山门前有两条路，路边各立一块石碑。赵晴儿指着石碑上的两个数说：「如果其中一个数能被另一个整除，这两块碑就是一对——走左边这条路。否则走右边。」

给定两个整数A和B，判断它们是否互为倍数关系（即A能被B整除，或B能被A整除）。

本题输出 Sao Multiplos / Nao sao Multiplos 为原题葡语格式（是倍数/不是倍数）。

class="io-section">

输入格式

一行，两个整数A和B。

输出格式

互为倍数输出 **Yes**，否则输出 **No**。

输入样例

6 24

输出样例

Sao Multiplos

class="hint"> 解题思路：

判断 $A \% B == 0$ 或 $B \% A == 0$ ，只要有一个成立就是倍数关系，输出Yes；否则输出No。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的输入输出操作

空间复杂度： $O(1)$

► 参考代码

JD016：三根木棍

简单 AcWing 664 | JD016

赵晴儿从柴堆里抽出三根木棍，量了量长度，让李少白判断它们能不能拼成一个三角形。如果能，算出周长；如果不能，算出以A、B为上下底、C为高的梯形面积，公式为 $(A+B) \times C \div 2$ 。

输入格式

一行，三个浮点数A、B、C。

输出格式

能拼成三角形输出 `Perimeter = X.X`（周长），拼不成输出 `Area = X.X`（梯形面积）。

输入样例

```
6.0 4.0 2.0
```

输出样例

```
Area = 10.0
```

解题思路：

三角形成立条件：任意两边之和大于第三边。成立时周长= $A+B+C$ ；不成立时梯形面积= $(A+B)*C/2$ 。保留1位小数。

复杂度分析：

时间复杂度： $O(1)$ — 直接公式计算

空间复杂度： $O(1)$

► 参考代码

JD017: 比武时辰

简单 AcWing 667 | JD017

李少白和师弟在练武场比武。他们从A时开始，到B时结束（只看整点，不看分钟）。如果 $B > A$ ，比武持续了 $B - A$ 小时；如果 $B \leq A$ ，说明比武过了零点，持续了 $24 - A + B$ 小时；如果 $A = B$ ，说明刚好持续了一整天（24小时）。

本题输出 `O JOGO DUROU X HORA(S)` 为原题葡语格式（游戏持续了X小时）。

class="io-section">

输入格式

一行，两个整数A和B ($0 \leq A, B \leq 23$)。

输出格式

输出一个整数，表示比武持续的小时数。

输入样例

16 2

输出样例

O JOGO DUROU 8 HORA(S)

class="hint"> 解题思路：

用条件判断：如果 $B > A$ ，输出 $B - A$ ；如果 $B = A$ ，输出24；否则输出 $24 - A + B$ 。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的比较操作

空间复杂度： $O(1)$

JD018：三石排序

简单 AcWing 663 | JD018

山道旁的空地上，梁嘉峰随手捡了三块石头，用刻刀在上面各凿了一个数字。他把石头往李少白脚前一丢：「乱的。从小到大排好，报给我。」李少白蹲下来，左右比了比。赵晴儿靠在树干上说：「先比前两个，把小的换到前面；再比后两个——这是冒泡排序最朴素的直觉。三个数只需要三次比较，但这个思路能推广到一万个数。」

输入格式

一行，三个整数，用空格隔开。

输出格式

三个数从小到大输出，空格隔开。

输入样例

```
7 21 -14
```

输出样例

```
-14 7 21
```

解题思路：

用条件判断两两比较确定顺序，或存入数组排序。

复杂度分析：

时间复杂度： $O(n \log n)$ — 排序算法

空间复杂度： $O(1)$

► 参考代码

JD019: 杂货铺

简单 AcWing 660 | JD019

山脚杂货铺有5种干粮，编号1到5，单价分别是4.00、4.50、5.00、2.00、1.50文。赵晴儿报了干粮编号和要买的数量，掌柜让她自己算总价。

输入格式

一行，两个整数：干粮编号X和数量Y。

输出格式

输出 `Total: R$` 后跟总价，保留两位小数。

输入样例

```
3 2
```

输出样例

```
Total: R$ 10.00
```

解题思路：

用数组或条件判断存储5种单价，查表后乘以数量。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的比较操作

空间复杂度： $O(1)$

► 参考代码

JD020：内力分级

简单 AcWing 659 | JD020

宗门把弟子的内力值分成四个等级：[0,25]是入门，(25,50]是初级，(50,75]是中级，(75,100]是高级。超出100则不在评级范围内。李少白测出一名弟子的内力值，需要判断他属于哪个等级。

输入格式

一个浮点数，表示内力值。

输出格式

输出对应区间的名称，或 `Out of interval`（超出范围）。

输入样例

```
95.29
```

输出样例

```
Interval (75,100]
```

解题思路：

用 if-elif-else 逐一判断落在哪个区间。

复杂度分析：

时间复杂度：O(1) — 固定次数的输入输出操作

空间复杂度：O(1)

► 参考代码

JD021: 沙盘点位

简单 AcWing 662 | JD021

演武堂的沙盘上铺着一张羊皮地图，横轴标着东——西，纵轴标着南——北。赵晴儿用朱砂点了一个红点 $P(x, y)$ ：「这是敌方哨探的位置。告诉我在哪个方位——第一象限是东北，第二象限是西北，第三象限是西南，第四象限是东南。如果刚好落在轴线上或者原点，也要指出来。」她指着沙盘边缘，「判断象限要看 x 和 y 的正负，先排除特殊位置——坐标轴和原点——再分象限，逻辑才清晰。」

输入格式

一行，两个浮点数 x 和 y 。

输出格式

输出 `Primeiro`（第一象限）、`Segundo`（第二）、`Terceiro`（第三）、`Quarto`（第四）、`Eixo X`（X轴上）、`Eixo Y`（Y轴上）或 `Origem`（原点）。

输入样例

4.5 -2.2

输出样例

Q4

解题思路：

先判断是否在原点或坐标轴上（ $x==0$ 或 $y==0$ ），再判断象限。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的输入输出操作

空间复杂度： $O(1)$

▶ 参考代码

JD022：跨夜比武

简单 AcWing 668 | JD022

李少白和师弟比武，开始时间是A时B分，结束时间是C时D分。比武可能跨了零点。请算出比武持续了多久。

输入格式

一行，四个整数A、B、C、D，分别表示开始的时、分和结束的时、分。

输出格式

输出比武持续时间，格式为 **H:M**。

输入样例

```
8 56 7 37
```

输出样例

```
22:41
```

解题思路：

全部转为分钟做差。如果结果 ≤ 0 ，加上 24×60 。然后转回时:分格式。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的输入输出操作

空间复杂度： $O(1)$

► 参考代码

JD023：账房调薪

简单 AcWing 669 | JD023

宗门账房年底调薪，按原月钱所在区间确定涨幅：0~400涨15%，400.01~800涨12%，800.01~1200涨10%，1200.01~2000涨7%，2000以上涨4%。涨幅是按整个工资乘以对应百分比，不是分段累进。

李少白拿到一名弟子的原月钱，请算出新月钱、涨了多少和涨幅百分比。

输入格式

一个浮点数，表示原月钱。

输出格式

三行：新月钱、涨了多少、涨幅百分比（整数带%号），金额保留两位小数。

输入样例

```
400.00
```

输出样例

```
New salary: 460.00
Increase: 60.00
Percentage: 15 %
```

解题思路：

用 if-elif 判断原月钱落在哪个区间，确定涨幅百分比p。涨额 = 原月钱 $\times$ p/100，新月钱 = 原月钱 + 涨额。注意400属于0~400区间（涨15%），400.01才属于400~800区间。

复杂度分析：

时间复杂度：O(1) — 固定次数的输入输出操作

空间复杂度: $O(1)$

▶ [参考代码](#)

JD024：五关考验

简单 AcWing 657 | JD024

宗门试炼场的石门前，立着一块铁碑，上面刻着五道关卡的通关条件。梁嘉峰拍了拍李少白的肩膀：「给你四个数A、B、C、D，要同时过这五关，门才会开。」他逐条念道：「第一，B大于A；第二，D大于A；第三，C与D之和大于A与B之和；第四，C必须是正数；第五，A必须是偶数。」赵晴儿补充道：「五个条件用逻辑与连接——任何一个不满足，门都不会开。这就是&&运算符的真正含义：全部为真，结果才为真。」

输入格式

一行，四个整数A、B、C、D。

输出格式

满足所有条件输出 `Accepted`，否则输出 `Not accepted`。

输入样例

```
-87 14 68 33
```

输出样例

```
Not accepted
```

解题思路：

用逻辑与（&&/and）连接五个条件，全部为真才通过。

复杂度分析：

时间复杂度：O(1) — 固定次数的输入输出操作

空间复杂度：O(1)

► 参考代码

JD025：信鸽传信

简单 AcWing 671 | JD025

宗门各分舵之间用信鸽传信，每个分舵有一个编号（DDD码）。李少白拿到一个编号，需要查出它对应哪个分舵：61=Brasilia，71=Salvador，11=Sao Paulo，21=Rio de Janeiro，31=Belo Horizonte 19=Campinas，其他编号则输出"DDD nao cadastrado"。

本题输出 DDD nao cadastrado 为原题葡语格式（未注册的区号）。

class="io-section">

输入格式

一个整数。

输出格式

输出对应分舵名称，或 `DDD nao cadastrado`。

输入样例

11

输出样例

Sao Paulo

解题思路：

用 if-elif 链逐一比对，或用字典/映射表查找。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的输入输出操作

空间复杂度： $O(1)$

► [参考代码](#)

JD026：木棍辨形

简单 AcWing 666 | JD026

梁嘉峰递来三根木棍：「先判断能不能拼成三角形。能的话，再判断是直角、锐角还是钝角三角形。」判断依据：最长边的平方与另外两边平方和的关系。

输入格式

一行，三个浮点数，表示三边长度（已从小到大排列）。

输出格式

不能构成输出 `Not a triangle`；能构成则输出三角形类型：`Equilateral`（等边）、`Isosceles`（等腰）、`Scalene`（不等边）、`Right`（直角）、`Acute`（锐角）、`Obtuse`（钝角）。

输入样例

```
6.0 4.0 2.0
```

输出样例

```
Not a triangle
```

解题思路：

先验证两边之和大于第三边。再判断等边/等腰/不等边，然后比较最长边平方判断直角/锐角/钝角。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的输入输出操作

空间复杂度： $O(1)$

▶ 参考代码

JD027：怪兽辨识

简单 AcWing 670 | JD027

迷踪林外遇到一只怪兽。赵晴儿观察了三层特征来判断它的种类：第一层是有脊椎还是无脊椎；第二层是哺乳类、鸟类还是爬行类；第三层是食性（食肉、食草、杂食）。

根据三层特征，输出对应的动物名称。

输入格式

三行，每行一个字符串，分别表示三层分类特征。

输出格式

输出对应的动物名称。

输入样例

```
vertebrado  
mamifero onivoro
```

输出样例

```
homem
```

解题思路：

三层嵌套 if-else：先判断第一层，再判断第二层，最后判断第三层。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的输入输出操作

空间复杂度： $O(1)$

► 参考代码

JD028：田赋计算

简单 AcWing 672 | JD028

李少白入了宗门，开始交田赋。税率分档：2000以下免税，2000.01~3000部分征8%，3000.01~4500部分征18%，4500以上部分征28%。注意是分段计税——只有超出部分按对应税率算。

本题输出 R\$ X.XX 和 Isento 为巴西货币和葡语免税标记。

class="io-section">

输入格式

一个浮点数，表示月收入。

输出格式

输出 R\$ X.XX。免税则输出 Isento。

输入样例

3002.00

输出样例

R\$ 80.36

解题思路：

分段计算： $1000 \times 8\% + 2 \times 18\% = 80.36$ 。每段只对超出部分征税。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的输入输出操作

空间复杂度: $O(1)$

▶ [参考代码](#)

第3章 千回百转

for与while

千层塔中，梁嘉峰指着重复的石阶："循环——让代码自动重复执行。掌握它，你就能用一行代码完成一百次操作。"

本章学习 for 循环和 while 循环——让程序自动重复执行任务。嵌套循环处理更复杂的问题。

JD029：石砖偶数

简单 AcWing ??? | JD029

千层塔第一层。墙上刻着一排编号从1到100的石砖。赵晴儿说：「把所有偶数编号的石砖挑出来，每行报一个。」

输入格式

无输入。

输出格式

输出全部偶数，每个偶数占一行。

输出样例

2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
48
50
52
54
56

58
60
62
64
66
68
70
72
74
76
78
80
82
84
86
88
90
92
94
96
98
100

class="hint"> 解题思路：

for循环从2开始，步长为2，到100结束。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

► 参考代码

JD030：奇数瓷砖

简单 AcWing ??? | JD030

千层塔第二层。墙上有一排编号从1到X的瓷砖。梁嘉峰指着瓷砖说：「把所有奇数编号的报出来。」

输入格式

一个正整数X。

输出格式

每行一个奇数，从1到X（含）。

输入样例

8

输出样例

1
3
5
7

解题思路：

for循环从1开始，步长为2，到X结束。如果X是偶数，最后一个奇数是X-1。

复杂度分析：

时间复杂度：O(n) — 单层循环遍历

空间复杂度：O(1)

► 参考代码

JD031: 反复之门

简单 AcWing ??? | JD031

千层塔的某一层有一扇不断重复的门。每次门前出现一个数X：如果X不为0，就从1数到X；如果X为0，修炼结束。

输入格式

若干行，每行一个整数X。以0结束。

输出格式

对每个非零的X，输出1到X，每行一个数。每组之间空一行。

输入样例

```
5
10
3
0
```

输出样例

```
1 2 3 4 5
1 2 3 4 5 6 7 8 9 10
1 2 3
```

解题思路：

外层while循环读入X，遇到0停止。内层for循环从1到X输出。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

▶ 参考代码

JD032：六奇连珠

简单 AcWing ??? | JD032

梁嘉峰递给李少白一个数X：「从X之后开始，找出6个连续的奇数，一个一个报给我。」

输入格式

一个整数X。

输出格式

从X之后开始的6个连续奇数，每行一个。

输入样例

8

输出样例

9
11
13
15
17
19

解题思路：

如果X是奇数，从X+2开始；如果X是偶数，从X+1开始。循环6次，每次加2。

复杂度分析：

时间复杂度：O(n) — 单层循环遍历

空间复杂度：O(1)

▶ 参考代码

JD033：正数清点

简单 AcWing ??? | JD033

千层塔第四层，石壁上嵌着六块刻度石板，每块显示一个数字——有的刻着正数，有的刻着负数。赵晴儿指着石板：「这是内力探测器的读数。正值说明经脉通畅，负值说明经脉淤堵。你帮我数数，通畅的有几条？」她顿了顿，「用循环逐个读入，遇到正数就累加计数器。这就是循环最常用的模式——遍历、判断、计数。」

输入格式

6行，每行一个浮点数。

输出格式

输出正数的个数，格式为 `x valores positivos`。

输入样例

```
7
-5
6
-3.4
4.6
12
```

输出样例

```
4 positive numbers
```

解题思路：

循环读入6个数，用if判断是否 >0 ，计数器累加。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

▶ [参考代码](#)

JD034：余数寻踪

简单 AcWing ??? | JD034

梁嘉峰写下了一个数 N ：「从1到10000，把所有除以 N 余数为2的数报给我。」

输入格式

一个整数 N ($N > 2$)。

输出格式

每行一个数，从1到10000中除以 N 余2的数。

输入样例

13

输出样例

2
15
28
41
54
...

解题思路：

for循环从1到10000，if $i \% N == 2$ 则输出。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

▶ 参考代码

JD035：区间计数

简单 AcWing ??? | JD035

药庐的柜台上，摊着N份药材的重量记录。赵晴儿用手指点了点：「这批药材，每份重量在10到20钱之间的才是合格品，太轻药效不足，太重则是掺了杂质。帮我分一分——合格的有几份，不合格的有几份？」梁嘉峰从门口探进头来：「循环读入每份数据，判断它是否落在区间内。两个计数器，一个记区间内，一个记区间外。这就是区间筛选的最基本思路。」

输入格式

第一行一个整数N。第二行N个整数。

输出格式

第一行输出 `X in`，X为区间内个数。第二行输出 `Y out`，Y为区间外个数。

输入样例

```
5
5 12 18 25 10
```

输出样例

```
3 in
2 out
```

解题思路：

循环读入，判断是否 $10 \leq X \leq 20$ ，计数器累加。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

▶ 参考代码

JD036：奇数求和

简单 AcWing ??? | JD036

梁嘉峰在墙上刻了两个数X和Y，让李少白算出X和Y之间（不含X和Y）所有奇数的和。

输入格式

一行，两个整数X和Y（X

输出格式

输出X和Y之间所有奇数的和。

输入样例

6 -5

输出样例

5

解题思路：

从X+1到Y-1循环，判断每个数是否为奇数，是则累加。

复杂度分析：

时间复杂度：O(n) — 单层循环遍历

空间复杂度：O(1)

► 参考代码

JD037：数链连珠

简单 AcWing ??? | JD037

梁嘉峰写下了一个起始整数A，又在第二行写了一串数。第二行里第一个正数就是N。请算出从A开始的N个连续整数的和。

输入格式

第一行：整数A。第二行：若干整数，第一个大于0的数才是N。

输出格式

从A开始N个连续整数之和。

输入样例

```
3
-5 0 -3 4 -1
```

输出样例

```
18
```

解题思路：

A固定，第二行跳过负数和零找N。从A开始循环N次累加。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

► 参考代码

JD038：石中之王

简单 AcWing ??? | JD038

练功场上摆了一排石块，每块上面刻着一个数。赵晴儿让李少白找到最大的数，并说出它是第几块（从1开始计数）。

输入格式

第一行一个整数 N ，表示石块数量。第二行 N 个整数。

输出格式

输出最大值和它的位置，格式见样例。

输入样例

```
5
3 2 5 1 4
```

输出样例

```
5
3
```

解题思路：

遍历时同时记录最大值和位置。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的比较操作

空间复杂度： $O(1)$

► 参考代码

JD039：约数寻踪

简单 AcWing ??? | JD039

藏经阁的书架前，赵晴儿翻开一本算经，在沙盘上写下一个数 N 。「约数，就是能整除这个数的数，」她解释道，「比如6的约数是1、2、3、6。找出一个数的所有约数，是数论的起点。」梁嘉峰坐在门槛上磨剑：「从1试到 N ，能整除的就是约数。循环加判断，就这么简单。」赵晴儿点点头：「先用最朴素的方法，以后再学优化——只试到 $\sqrt{N}$ 就够了。」

输入格式

一个正整数 N 。

输出格式

每行一个约数，从小到大。

输入样例

6

输出样例

1
2
3
6

解题思路：

从1到 N 循环，如果 $N \% i == 0$ 则 i 是约数。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

▶ 参考代码

JD040：九九归真

简单 AcWing ??? | JD040

赵晴儿教李少白背乘法口诀。她写下一个数N，让李少白从1乘到10，按格式列出N的乘法表：
「 $i \times N = i * N$ 」。

输入格式

一个整数N。

输出格式

10行，格式为 $i \times N = i * N$ 。

输入样例

140

输出样例

```
1 x 140 = 140
2 x 140 = 280
3 x 140 = 420
4 x 140 = 560
5 x 140 = 700
6 x 140 = 840
7 x 140 = 980
8 x 140 = 1120
9 x 140 = 1260
10 x 140 = 1400
```

解题思路：

for循环从1到10，每次输出 $i \times N = i * N$ 。

复杂度分析:

时间复杂度: $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

▶ [参考代码](#)

JD041: 千层阵列

简单 AcWing ??? | JD041

千层塔的一面墙上刻着N行M列的数字阵列，每行从1开始递增，但每逢第M个位置不写数字，写SWORD。赵晴儿让李少白按这个规律把整个阵列报出来。

输入格式

一行，两个整数N和M。

输出格式

N行，每行M个值，最后一个用SWORD代替，值之间用空格隔开。

输入样例

```
7 4
```

输出样例

```
1 2 3 SWORD
5 6 7 SWORD
9 10 11 SWORD
13 14 15 SWORD
17 18 19 SWORD
21 22 23 SWORD
25 26 27 SWORD
```

解题思路：

嵌套循环，外层控制行，内层控制列。每行第M个位置输出SWORD。

复杂度分析：

时间复杂度： $O(n^2)$ — 嵌套循环

空间复杂度： $O(1)$

▶ 参考代码

JD042：兵刃统计

简单 AcWing ??? | JD042

演武场上，弟子们正在操练。李少白走过一排兵器架，记录了N次观察：每次看到一件兵器，C代表剑，R代表刀，F代表枪。赵晴儿让他统计每种兵器出现了多少把，以及各自的占比。

输入格式

第一行一个整数N。接下来N行，每行一个整数（数量）和一个字符（C/R/F），分别表示某次看到的兵器数量和类型。

输出格式

第一行输出 `Total: X weapons`（总数量）。接下来三行输出每种兵器的总数：`Total swords: X`、`Total blades: X`、`Total spears: X`。最后三行输出各自占比：`Percentage of swords: XX.XX %` 等，保留两位小数。

输入样例

```
11
1 F
5 C
4 C
12 C
11 F
15 F
2 F
7 C
1 C
4 C
9 F
```

输出样例

```
Total: 71 weapons
Total swords: 33
Total blades: 0
Total spears: 38
Percentage of swords: 46.48 %
```

Percentage of blades: 0.00 %
Percentage of spears: 53.52 %

解题思路：

三个计数器分别累加C、R、F的数量。注意每行先读数量再读类型。总数 = 所有数量之和。占比 = 各类数量 / 总数 $\times 100\%$ ，保留两位小数。

复杂度分析：

时间复杂度： $O(1)$ — 固定次数的输入输出操作

空间复杂度： $O(1)$

► 参考代码

第4章 排兵布阵

数组

赵晴儿在沙盘上排了一列石块："数组——用一个名字管理一整排数据。"

本章学习数组——存储和管理一组有序数据。遍历、求和、排序，这些操作将伴随你整个编程生涯。

JD043：正剑除邪

简单 AcWing JD | JD043

练功房的石板上有10道剑气留下的痕迹，每道痕迹上刻着一个数。其中有些数带着邪气（负数或零），梁嘉峰让李少白运起纯阳内力，把这些不纯正的数全部净化，替换成1——象征正气归于一元。

输入格式

一行，10个整数，用空格隔开。

输出格式

替换后的10个数，每行格式为 `x[i] = 值`。

输入样例

```
-73
27
-35
50
-67
86
39
-81
-7
31
```

输出样例

```
X[0] = 1
X[1] = 27
X[2] = 1
X[3] = 50
X[4] = 1
X[5] = 86
X[6] = 39
X[7] = 1
X[8] = 1
```

`x[9] = 31`

解题思路：

遍历数组，将 ≤ 0 的元素替换为1。

复杂度分析：

时间复杂度： $O(n)$ — 遍历数组

空间复杂度： $O(1)$

► **参考代码**

JD044：翻倍剑阵

简单 AcWing JD | JD044

赵晴儿在沙盘上布下一个翻倍剑阵。她说：「以第一个数为起点，后面每个数都是前一个的两倍。内力如此，出剑亦是如此——一重更比一重强。」

输入格式

一个整数 V 。

输出格式

10行，格式为 $N[i] = x$ 。

输入样例

6

输出样例

```
N[0] = 6
N[1] = 12
N[2] = 24
N[3] = 48
N[4] = 96
N[5] = 192
N[6] = 384
N[7] = 768
N[8] = 1536
N[9] = 3072
```

解题思路：

$N[0]=V$ ，循环 $N[i]=N[i-1]*2$ 。

复杂度分析:

时间复杂度: $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

► [参考代码](#)

JD045：剑中取精

简单 AcWing JD | JD045

沙盘上散落着100把长剑，每把剑上刻着一个数，代表剑的品级。赵晴儿让李少白从中挑出品级不超过10的精良之剑，逐一记录。

输入格式

100个浮点数。

输出格式

按顺序输出所有 ≤ 10 的元素，每行格式为 `A[i] = x`（保留一位小数）。

输入样例

```
79.5
-35.1
65.0
-89.9
62.2
-30.0
-58.1
-74.0
85.7
66.4
-60.2
-32.1
11.4
34.6
0.2
96.3
-67.9
70.1
27.3
53.4
63.8
43.2
-68.2
-87.9
-13.3
62.1
-12.0
66.9
```

1.6
-70.3
-14.0
-84.8
19.6
-1.9
43.4
36.9
97.2
-88.6
52.2
-21.0
-54.1
-17.3
-11.7
-8.5
40.9
-21.6
-12.0
87.8
-92.9
45.3
-70.1
73.7
-23.2
87.3
-78.6
38.2
51.2
-21.0
3.9
88.4
28.3
-67.6
47.1
-49.1
73.6
39.5
28.0
-80.1
-41.4
43.2
-4.6
61.1
83.2
65.5
74.3
90.1
-80.2
39.6
-7.0
-34.4
43.1

-62.5
-25.3
61.0
68.5
9.0
10.3
45.9
-98.0
-34.8
88.3
-26.6
-48.1
20.5
-5.4
-76.7
79.7
-90.0
59.1
-18.7

输出样例

A[1] = -35.1
A[3] = -89.9
A[5] = -30.0
A[6] = -58.1
A[7] = -74.0
A[10] = -60.2
A[11] = -32.1
A[14] = 0.2
A[16] = -67.9
A[22] = -68.2
A[23] = -87.9
A[24] = -13.3
A[26] = -12.0
A[28] = 1.6
A[29] = -70.3
A[30] = -14.0
A[31] = -84.8
A[33] = -1.9
A[37] = -88.6
A[39] = -21.0
A[40] = -54.1
A[41] = -17.3
A[42] = -11.7
A[43] = -8.5
A[45] = -21.6
A[46] = -12.0
A[48] = -92.9
A[50] = -70.1

```
A[52] = -23.2
A[54] = -78.6
A[57] = -21.0
A[58] = 3.9
A[61] = -67.6
A[63] = -49.1
A[67] = -80.1
A[68] = -41.4
A[70] = -4.6
A[76] = -80.2
A[78] = -7.0
A[79] = -34.4
A[81] = -62.5
A[82] = -25.3
A[85] = 9.0
A[88] = -98.0
A[89] = -34.8
A[91] = -26.6
A[92] = -48.1
A[94] = -5.4
A[95] = -76.7
A[97] = -90.0
A[99] = -18.7
```

解题思路：

遍历数组，if元素 ≤ 10 则输出。

复杂度分析：

时间复杂度： $O(n)$ — 遍历数组

空间复杂度： $O(1)$

► 参考代码

JD046：剑阵倒转

简单 AcWing JD | JD046

夕阳西斜，练功场上二十把长剑按品级从低到高排成一列，剑锋朝东。梁嘉峰走过来，一脚踢翻了沙盘旁的水壶，水流了一地。

「少白，你布的这个剑阵，后排的精锐反而离敌人最远。」他蹲下来，在泥地上画了两个箭头，「攻守之道，有时需要颠倒乾坤——让最弱的顶在前面当诱饵，最强的藏在后方一击致命。」

李少白挠挠头：「那……第一把和最后一把交换，第二把和倒数第二把交换？」

梁嘉峰拍了拍他的肩膀：「没错。双指夹剑，首尾互换，一招之内，阵势逆转。去练吧。」

输入格式

20个整数。

输出格式

翻转后的20个数，每行格式为 `N[i] = X`。

输入样例

```
-40
57
99
2
-20
25
-67
-76
70
98
-95
-92
-100
-100
-55
48
-55
54
1
```

-32

输出样例

```
N[0] = -32
N[1] = 1
N[2] = 54
N[3] = -55
N[4] = 48
N[5] = -55
N[6] = -100
N[7] = -100
N[8] = -92
N[9] = -95
N[10] = 98
N[11] = 70
N[12] = -76
N[13] = -67
N[14] = 25
N[15] = -20
N[16] = 2
N[17] = 99
N[18] = 57
N[19] = -40
```

解题思路：

双指针从两端向中间交换。

复杂度分析：

时间复杂度： $O(n + m)$ — 双指针遍历

空间复杂度： $O(1)$

► 参考代码

JD047：剑锋所指

简单 AcWing JD | JD047

演武场的东侧，一排高低不一的木桩深深扎在泥土里。晨露顺着木桩的纹路滑落，在最低的那根旁边积了一小滩水洼。

赵晴儿站在木桩阵前，长发被晨风吹得微微飘动。她拔出腰间短剑，剑尖指向最矮的那根木桩：「少白，剑道之中有一招叫「以弱制动」——找到对方最薄弱的一点，一剑刺入，满盘皆活。」

她转身看向李少白：「这排木桩上各刻着一个数。找到最小的那个数，告诉我它在第几根桩上。注意——编号从0开始。」

李少白蹲下身，一根一根看过去。赵晴儿摇头：「用眼睛找太慢，万一有一千根呢？遍历一遍，记住当前最小值和它的位置——这就是「剑锋所向」。」

本题输出 Menor valor / Posicao 为原题葡语格式（最小值/位置）。

class="io-section">

输入格式

第一行一个整数N。第二行N个整数。

输出格式

输出最小值和它的位置，格式见样例。

输入样例

```
10
1 2 3 4 -5 6 7 8 9 10
```

输出样例

```
Menor valor: -5
Posicao: 4
```

解题思路：

遍历时记录最小值和位置。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

► 参考代码

JD048：二气相生

简单 AcWing JD | JD048

赵晴儿在沙盘上画出两道气旋：「阴阳二气，交替相生。第一气为0，第二气为1，此后每气皆为前二气之和。只用两个变量，算出第N气。」这就是斐波那契数列。

输入格式

一个整数N。

输出格式

输出F(0)到F(N)的所有值，空格隔开。

输入样例

4

输出样例

3

解题思路：

两个变量滚动： $a, b = b, a + b$ 。

复杂度分析：

时间复杂度： $O(n)$ — 线性递推，两个变量滚动计算

空间复杂度： $O(n)$

► 参考代码

JD049：斐波剑诀

简单 AcWing JD | JD049

赵晴儿展开一卷古剑谱，上面记载着斐波那契剑诀：0, 1, 1, 2, 3, 5, 8, 13……「每一式都是前两式的融合。T次询问，每次报出第N式。」

输入格式

第一行一个整数T。接下来T行，每行一个整数N。

输出格式

每行输出对应的F(N)。

输入样例

```
5
```

输出样例

```
0 1 1 2 3
```

解题思路：

预处理斐波那契数组到60，查询时直接取值。

复杂度分析：

时间复杂度：O(n) — 预处理斐波那契数组，查询O(1)

空间复杂度：O(n)

► 参考代码

JD050：列阵求和

简单 AcWing JD | JD050

清晨的练功场还带着露水的凉意。梁嘉峰蹲在沙盘前，用树枝刻下两个数字：一个起点M，一个终点N。

「少白，江湖中有一种内功心法，需要从第M式练到第N式，每一式都要走一遍，最后统计总消耗的内力值。」他抬起头，目光锐利，「不过有个规矩——如果起手式比收手式的编号还大，说明你记反了，得先翻过来再练。」

李少白蹲下身，认真地在石板上从M一路刻到N：「师兄，那每一式的内力消耗就是它的编号本身？」

梁嘉峰点头：「列阵，求和，练完一组再来下一组，直到你输入两个零为止。去吧。」

输入格式

若干行，每行两个整数M和N。以两个0结束。

输出格式

每组：先输出序列（空格隔开），再输出 `Sum=S` 。

输入样例

```
5 10
2 3
0 0
```

输出样例

```
5 6 7 8 9 10 Sum=45
2 3 Sum=5
```

✦ **解题思路：**

循环从M到N输出并累加。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

► **参考代码**

JD051: 剑行取道

简单 AcWing JD | JD051

夜色渐浓，演武场中央的十二宫剑阵亮着幽幽的蓝光。赵晴儿盘膝坐在阵前，发丝被晚风吹得微微飘动。

「少白，你看这剑阵，横十二剑，纵十二剑，共一百四十四把。」她伸出纤手指向其中一行，「每把剑的力量强弱不同。有时候，我们不需要知道整个阵的全貌——只需看清一行的走势，就能判断敌人的主攻方向。」

李少白凑近细看，每个格子里都跳动着—个浮点数，有的正，有的负，如同内力有强有弱。

「我给你一个行号 L ，再给你一个指令—— S 是求这一行的内力总和， M 是求平均值。」赵晴儿微微一笑，「剑道修行，既要能纵览全局，也要能聚焦—处。去算吧。」

输入格式

第一行—个整数 L （ $0 \sim 11$ ，表示行号）。第二行—个字符（ S 或 M ）。接下来12行，每行12个浮点数。

输出格式

输出该行的和或平均值，保留—位小数。

输入样例

```
1
M
94.2 65.35 7.41 -67.85 18.62 -72.49 -5.91 66.06 -20.35 -17.4 12.62 -63.04
...
```

输出样例

```
17.3
```



解题思路：

遍历指定行，根据操作类型求和或求平均。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

► 参考代码

JD052：剑列纵横

简单 AcWing JD | JD052

第二天清晨，赵晴儿又把李少白带到了剑阵前。晨雾还没散尽，剑阵的蓝光在雾气中显得格外迷离。

「昨天你算的是横行之力，今天换个方向。」她沿着一列从上往下划了一道，「剑阵之中，行有行势，列有列气。有时候敌人不会从正面攻来，而是沿着纵列渗透——比如顺着第三把剑的位置，从第一行一直打到最后一行。」

李少白恍然：「所以要算一列的总力量，才能知道哪条纵线最薄弱？」

「孺子可教。」赵晴儿递给他一个新的列号C和指令，「S求和，M求平均。方向变了，思路还是一样的——遍历每一行，取出同一列的元素。」

输入格式

第一行一个整数C（0~11，表示列号）。第二行一个字符（S或M）。接下来12行，每行12个浮点数。

输出格式

输出该列的和或平均值，保留一位小数。

输入样例

```
1
S
-29.7 94.5 43.8 3.7 68.1 19.9 23.4 -0.3 -85.3 57.3 89.7 36.0
...
```

输出样例

```
199.5
```

✦ **解题思路：**

遍历所有行取指定列元素。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

► **参考代码**

JD053：完璧归宗

简单 AcWing JD | JD053

赵晴儿在沙盘上写下「完璧」二字：「完璧之数，等于它的所有真因子之和。譬如 $6=1+2+3$ 。找到不超过N的所有完璧之数，以印证归宗之意。」

输入格式

一个整数N。

输出格式

每行一个完全数，格式为 $x = f1 + f2 + f3 + \dots + fk$ ，所有真因子从小到大用加号连接。

输入样例

30

输出样例

6
28

解题思路：

对每个数找所有真因子求和，等于自身则输出。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历矩阵元素

空间复杂度： $O(n^2)$

► 参考代码

JD054：剑意之质

简单 AcWing JD | JD054

梁嘉峰在沙盘上点出几枚剑棋：「天下剑法，有些可以拆分化简，有些不可再分，乃剑意之基石。这些不可拆分的棋位——只能被1和自己整除的数——就是剑意之质。排出2到N之间所有的剑意质数。」

输入格式

一个整数N。

输出格式

输出2到N之间所有的质数，每行一个。

输入样例

10

输出样例

2
3
5
7

解题思路：

对每个数从2到 $\sqrt{n}$ 试除。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历矩阵元素

空间复杂度： $O(n^2)$

▶ 参考代码

第5章 十二宫剑阵

二维数组

十二宫剑阵展开，光幕上浮现一个 12×12 的方格。"二维数组——矩阵。行与列交织，每个格子都是一个数据点。"

本章学习二维数组——矩阵的存储、遍历和方向数组。掌握这些，你就能操控任何网格结构。

JD055：上方剑域

简单 AcWing JD | JD055

演武场上空，赵晴儿凌空画出一道 12×12 的剑气光幕。剑阵分十二宫方位，她指向光幕上方：

「上域之剑，取天位之力。首行的剑主攻正面，越往下剑气越弱。算算上域之力和。」

输入格式

第一行一个大写字母S或M。接下来12行，每行12个浮点数。

输出格式

输出对应区域的和或平均值，保留一位小数。

输入样例

```
S
5.0 65.0 82.0 54.0 60.0 96.0 64.0 88.0 81.0 2.0 86.0 29.0
66.0 16.0 27.0 63.0 1.0 52.0 67.0 76.0 94.0 88.0 65.0 39.0
48.0 61.0 15.0 22.0 48.0 30.0 57.0 15.0 97.0 18.0 19.0 49.0
...
```

输出样例

```
1594.0
```

解题思路：

先读操作类型（S求和/M求平均），再判断行号。上方区域即 i

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历矩阵元素

空间复杂度： $O(n^2)$

▶ 参考代码

JD056：下方剑域

简单 AcWing JD | JD056

「下域之剑，承地脉之气。」

赵晴儿的手指向光幕下方划去，

「靠近大地的剑势更沉。算算下域之力和。」

输入格式

第一行一个大写字母S或M。接下来12行，每行12个浮点数。

输出格式

输出对应区域的和或平均值，保留一位小数。

输入样例

```
S
6.0 90.0 70.0 55.0 69.0 69.0 62.0 79.0 7.0 9.0 54.0 43.0
30.0 75.0 74.0 58.0 99.0 52.0 88.0 51.0 77.0 92.0 77.0 10.0
89.0 99.0 84.0 60.0 90.0 25.0 95.0 82.0 96.0 5.0 92.0 10.0
...
```

输出样例

```
1694.0
```

解题思路：

下方区域即 $i > j$ 的元素。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历矩阵元素

空间复杂度: $O(n^2)$

▶ [参考代码](#)

JD057：左方剑域

简单 AcWing JD | JD057

「左域之剑，守青龙之位。」

赵晴儿指向光幕左侧，

「左方剑阵主守，去势较短。算算左域之力和。」

输入格式

第一行一个大写字母S或M。接下来12行，每行12个浮点数。

输出格式

输出对应区域的和或平均值，保留一位小数。

输入样例

```
S
68.0 59.0 10.0 88.0 95.0 50.0 60.0 8.0 6.0 7.0 5.0 79.0
18.0 94.0 63.0 2.0 15.0 13.0 39.0 37.0 33.0 27.0 11.0 29.0
12.0 23.0 6.0 42.0 4.0 63.0 7.0 20.0 11.0 74.0 27.0 5.0
...
```

输出样例

```
1410.0
```

解题思路：

左方区域即满足 $j < i$ & & $i + j < 11$ 的元素（主对角线下且反对角线上）。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历矩阵元素

空间复杂度: $O(n^2)$

▶ [参考代码](#)

JD058: 右方剑域

简单 AcWing JD | JD058

「右域之剑，据白虎之位。」

赵晴儿指向光幕右侧，

「右方剑阵主攻，去势较长。算算右域之力和。」

输入格式

第一行一个大写字母S或M。接下来12行，每行12个浮点数。

输出格式

输出对应区域的和或平均值，保留一位小数。

输入样例

```
S
69.0 84.0 98.0 42.0 4.0 23.0 58.0 99.0 32.0 14.0 26.0 40.0
29.0 6.0 10.0 50.0 60.0 13.0 7.0 59.0 16.0 31.0 23.0 0.0
0.0 61.0 75.0 80.0 46.0 58.0 45.0 34.0 10.0 61.0 53.0 66.0
...
```

输出样例

```
1203.0
```

解题思路：

右方区域即满足 $j > i$ & $i + j > 11$ 的元素（主对角线上且反对角线下）。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历矩阵元素

空间复杂度: $O(n^2)$

▶ [参考代码](#)

JD059: 右上剑角

简单 AcWing JD | JD059

「右上剑角——天乾之位，日升之处。」

赵晴儿在沙盘上比划，

「从主对角线往上，剑势腾空而起，覆盖右上半壁。算算这角的剑气总和。」

输入格式

第一行一个大写字母S或M。接下来12行，每行12个浮点数。

输出格式

输出右上半部分的和或平均值，保留一位小数。

输入样例

```
S
86.0 26.0 52.0 44.0 73.0 60.0 63.0 26.0 54.0 1.0 35.0 0.0
60.0 49.0 95.0 82.0 59.0 59.0 69.0 19.0 72.0 79.0 82.0 75.0
86.0 23.0 26.0 13.0 5.0 99.0 13.0 84.0 28.0 26.0 60.0 23.0
...
```

输出样例

```
3319.0
```

解题思路：

右上部分即 $j > i$ 的元素。S求和，M求平均。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历矩阵元素

空间复杂度: $O(n^2)$

▶ [参考代码](#)

JD060: 左上剑角

简单 AcWing JD | JD060

「左上剑角——天坤之位，月落之处。从主对角线往上左方，剑气如新月之弧，划出一角天地。」

输入格式

第一行一个大写字母S或M。接下来12行，每行12个浮点数。

输出格式

输出对应区域的和或平均值，保留一位小数。

输入样例

```
S
49.0 20.0 80.0 78.0 8.0 14.0 59.0 46.0 32.0 6.0 7.0 97.0
59.0 80.0 31.0 74.0 20.0 20.0 88.0 27.0 11.0 18.0 28.0 12.0
97.0 85.0 70.0 33.0 61.0 32.0 10.0 89.0 42.0 82.0 21.0 79.0
...
```

输出样例

```
2981.0
```

解题思路：

左上部分即 $i+j<11$ 的元素。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历矩阵元素

空间复杂度： $O(n^2)$

▶ 参考代码

JD061: 右下剑角

简单 AcWing JD | JD061

「右下剑角——地乾之位，日落之处。从主对角线往下右方，剑势如残阳下沉，渐行渐远。」

输入格式

第一行一个大写字母S或M。接下来12行，每行12个浮点数。

输出格式

输出对应区域的和或平均值，保留一位小数。

输入样例

```
S
80.0 49.0 14.0 87.0 99.0 61.0 17.0 39.0 57.0 46.0 63.0 43.0
79.0 19.0 8.0 27.0 77.0 77.0 97.0 82.0 52.0 14.0 85.0 36.0
61.0 75.0 96.0 20.0 78.0 6.0 1.0 72.0 15.0 46.0 23.0 98.0
...
```

输出样例

```
2790.0
```

解题思路：

右下部分即满足 $i + j > 11$ 的元素（反对角线以下）。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历矩阵元素

空间复杂度： $O(n^2)$

► 参考代码

JD062: 左下剑角

简单 AcWing JD | JD062

「左下剑角——地坤之位，月升之处。从主对角线往下左方，剑气似初月升空，一弧清辉。」

输入格式

第一行一个大写字母S或M。接下来12行，每行12个浮点数。

输出格式

输出对应区域的和或平均值，保留一位小数。

输入样例

```
S
43.0 96.0 42.0 21.0 34.0 15.0 13.0 59.0 82.0 4.0 35.0 93.0
78.0 97.0 91.0 19.0 38.0 91.0 16.0 90.0 38.0 53.0 31.0 73.0
25.0 37.0 40.0 40.0 34.0 39.0 98.0 77.0 29.0 2.0 84.0 54.0
...
```

输出样例

```
3154.0
```

解题思路：

左下部分即满足 $j < i$ 的元素（主对角线以下）。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历矩阵元素

空间复杂度： $O(n^2)$

► 参考代码

JD063: 方圆初阵

简单 AcWing JD | JD063

「此乃方圆剑阵——最外圈剑气最淡，标记为1；往里一圈剑气渐浓，标记为2。」

赵晴儿在沙盘上画出一圈圈剑芒，

「到剑阵中心，剑气愈凝愈实。每一圈剑芒的强度，就是你离外界最远的那道剑气的距离。」

输入格式

多行，每行一个整数 N 。 $N=0$ 时结束。

输出格式

每个 N 输出一个 N 阶矩阵，每个数字占3个字符宽度。每个矩阵后跟一个空行。

输入样例

```
1
0
```

输出样例

```
1
```

解题思路：

每个位置的值 = $\min(i+1, j+1, N-i, N-j)$ ，即到四条边的最小距离。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历 $n \times n$ 矩阵， $O(1)$ 填充每个位置

空间复杂度： $O(n^2)$

▶ 参考代码

JD064：镜像方阵

简单 AcWing JD | JD064

「此阵以主对角线为镜——镜中剑影，左右互映。」

赵晴儿落下第一柄长剑，

「主对角线上的剑势最强，为1；往右上和左下方逐次递减，如剑光在水面的倒影越远越淡。」

输入格式

多行，每行一个整数 N 。 $N=0$ 时结束。

输出格式

每个 N 输出一个 N 阶矩阵，每个数字占3个字符宽度。每个矩阵后跟一个空行。

输入样例

```
1
0
```

输出样例

```
1
```

解题思路：

$M[i][j] = |i-j| + 1$ 。对角线为1，离对角线越远数越大。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历矩阵元素

空间复杂度： $O(n^2)$

▶ 参考代码

JD065：倍增方阵

简单 AcWing JD | JD065

「此阵每一格，剑气强度皆由剑气原点(0,0)倍增而来。」

赵晴儿剑尖轻点沙盘，

「位在(i,j)的剑势，强度为2的(i+j)次方。一生二，二生四，剑气倍增如潮水蔓延。」

输入格式

多行，每行一个整数 N ($0 \leq N \leq 15$)。 $N=0$ 时结束。

输出格式

每个 N 输出一个 N 阶矩阵，每行 N 个整数用空格隔开。每个矩阵后跟一个空行。

输入样例

```
3 0
```

输出样例

```
1 2 4
2 4 8
4 8 16
```

解题思路：

$M[i][j] = 2^{(i+j)}$ 。用位运算 $1 \ll (i+j)$ 或 pow 函数。

复杂度分析：

时间复杂度： $O(n^2)$ — 遍历 $n \times n$ 矩阵，位运算计算幂次 $O(1)$

空间复杂度： $O(n^2)$

▶ 参考代码

JD066: 蛇形剑阵

简单 AcWing JD | JD066

「剑阵蜿蜒如蛇，以身引气，气走周身。」

赵晴儿在沙盘上画了一个 n 行 m 列的剑阵，

「从左上角开始，依次填入从1到 $n \times m$ 的剑气。行到尽头便拐弯，似灵蛇折返。」

输入格式

一行，两个整数 n 和 m 。

输出格式

n 行，每行 m 个整数用空格隔开。输出后跟一个空行。

输入样例

```
3 3
```

输出样例

```
1 2 3
8 9 4
7 6 5
```

解题思路：

模拟蛇形遍历：右→下→左→上循环。维护四个边界。

复杂度分析：

时间复杂度： $O(n^2)$ — 蛇形遍历矩阵，每个元素访问一次

空间复杂度： $O(n^2)$

▶ 参考代码

第6章 古卷密文

字符串处理

夜风卷起藏经阁的竹帘，月光洒在一排排古旧的竹简上。赵晴儿手持灯笼，缓步走过书架，指尖拂过积满灰尘的卷轴。

「少白，剑道的传承不在刀剑之中，而在文字之间。」她抽出一卷泛黄的密文，展开铺在书案上，「字符串——字符的序列。古人把毕生所学刻在竹简上，一字一句，皆是心血。读懂文字，才能读懂剑意。」

本章学习字符串处理——查找、替换、分割、加密。这些操作看似枯燥，却是破解古卷密文、传承剑道精髓的钥匙。

JD067: 古卷测长

简单 AcWing JD | JD067

藏经阁的木架上积了薄薄一层灰。梁嘉峰从最高处取下一卷竹简，吹去上面的尘土，递到李少白手里。竹简入手冰凉，上面密密麻麻刻着一行字，有空格，有标点。

「少白，修习剑道的第一步，是认清你面前有多少敌人。」梁嘉峰靠在书架上，双臂抱胸，「这卷竹简上刻了多少个字符——包括空格、标点，一个都不能少。数不清长度，就谈不上读懂内容。」

李少白展开竹简，手指一个一个点过去。梁嘉峰摇摇头：「用眼睛数太慢。在代码里，一行字符串的长度，一个函数就能告诉你。」

输入格式

一行字符串（长度不超过100），可能包含空格。

输出格式

输出字符串的实际长度。

输入样例

```
Hello World
```

输出样例

```
11
```

解题思路：

读入一行字符串（含空格），输出其长度。

复杂度分析:

时间复杂度: $O(n)$ — 线性遍历处理

空间复杂度: $O(n)$

► [参考代码](#)

JD068：密文留白

简单 AcWing JD | JD068

烛火摇曳，赵晴儿展开一卷年代久远的密文。字迹潦草，笔画相连，几乎分不清哪里是上一个字的收笔，哪里是下一个字的起笔。

「少白，你看这卷密文——古人为了节省竹简，字与字之间不留空隙。但我们要逐字破译，必须先把它们分开。」她拿起朱砂笔，在第一个字后面点了一个红点，「每个字符后面加一个空格，这样每个字就独立出来了。」

李少白接过笔，正要一个一个点下去，赵晴儿按住他的手：「别用手抄。密文可能有上百个字，逐个加空格这种机械活，交给代码去做。遍历字符串，每个字符后面输出一个空格——简单，但别忘了最后一个字符后面也要加。」

输入格式

一行字符串（长度不超过100）。

输出格式

每个字符后加一个空格输出（最后一个字符后也有空格）。

输入样例

abc

输出样例

a b c

解题思路：

遍历字符串，每个字符后输出一个空格。

复杂度分析:

时间复杂度: $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

► [参考代码](#)

JD069: 墨迹遮字

简单 AcWing JD / JD069

李少白从箱底翻出一卷残破的古卷，上面的字迹被岁月侵蚀得斑驳陆离。有些字还清晰可辨，有些却被大片墨渍吞没，只留下一团团漆黑的污迹。

赵晴儿凑过来看了一眼：「这卷密文里，有一种特定的字符被敌人故意用墨汁涂抹了。我们要做的不是修复它，而是标记它——把所有那个字符都替换成'#'，这样既保留了原文的结构，又能一眼看出哪里被篡改过。」

她从袖中取出一根细竹签，蘸了朱砂：「你告诉我哪个字符被遮了，我就用'#'盖住它。不过在代码里，这件事更简单——遍历字符串，遇到目标字符就替换，其他的原样保留。」

输入格式

一行字符串（长度不超过100，全大写）。第二行一个字符，表示要替换的字符。

输出格式

替换后的字符串。

输入样例

```
hello
o
world
l
abc
x
test
e
hello
a
h
e
```

输出样例

```
hell#
```

解题思路：

遍历字符串，遇到目标字符输出'#'，否则输出原字符。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

► 参考代码

JD070: 经文重构

简单 AcWing JD | JD070

月光透过窗棂洒在书案上，赵晴儿摊开一卷泛黄的经文，指尖轻轻划过上面的文字。

「少白，古人的加密术比你想象的精妙。这卷经文a看似普通，但真正的密文b藏在字与字的缝隙里。」她取出一张白纸，写下第一条规则，「b的每一个字，取的是a中相邻两个字的内力之和——把两个字符的ASCII码加在一起，再转回字符，就是b对应位置的密文。」

李少白瞪大眼睛：「那最后一个字呢？它后面没有字了啊。」

赵晴儿赞许地点头：「问得好。古人的智慧在于循环——最后一个字要和第一个字配对，首尾相连，如同太极阴阳。去写吧，这叫「经文重构」。」

输入格式

一行字符串a（长度3~100）。

输出格式

构造后的字符串b。

输入样例

```
--9+=.3'<<-!!'+5.$#:)6*(6.(;7>*)>35#5.72#/44;/)0#864"!51$1=00*:$&'
```

输出样例

```
jfdhkaZcxiNBHR`cjklhZbimiqxobc^caZUO^mbG]c_`R^dVcruhSgqhXXceiURchojXYS[njVCVfUUnm`.
```

解题思路：

遍历a， $b[i] = \text{chr}(\text{ord}(a[i]) + \text{ord}(a[i+1]))$ 。最后一个用a[-1]和a[0]。

复杂度分析:

时间复杂度: $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

► [参考代码](#)

JD071: 密文数符

简单 AcWing JD | JD071

藏经阁的烛火映着赵晴儿的侧脸，她正在翻阅一卷从西域传来的密文。密文上文字与数字交织缠绕，如同乱麻。

「少白，这卷密文里藏着一组关键的数字密码——但它们混在普通文字里，肉眼难以分辨。」她把密文铺在桌上，「破译的第一步，是数清里面有几个数字字符——'0'到'9'，每一个都可能是打开机关的钥匙。」

李少白凑近细看，手指在密文上一个字一个字地点过：「这个是字母……这个是数字……这个也是……」

赵晴儿轻敲桌面：「别用手。遍历字符串，判断每个字符是否在'0'到'9'之间，是就计数加一。这叫「密文数符」——数字和符号的统计，是密码学的基本功。」

输入格式

一行字符（长度不超过100）。

输出格式

输出其中数字字符（'0'~'9'）的个数。

输入样例

```
hello2024world
```

输出样例

```
4
```

解题思路：

遍历字符串，判断每个字符是否在'0'到'9'之间。

复杂度分析:

时间复杂度: $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

► [参考代码](#)

JD072：拳掌对决

简单 AcWing JD | JD072

午后的练功场难得清闲。李少白和师弟蹲在树荫下，满头大汗地比划着拳头。梁嘉峰路过，停下脚步。

「你俩在干嘛？」

「猜拳！师兄来评评理——」师弟嚷道，「我出石头他出布，他说他赢。可我觉得石头更厉害！」

梁嘉峰忍不住笑了：「剑道之中，没有绝对的强弱，只有相生相克。石头砸剪刀，剪刀剪布，布包石头——循环往复，谁也不比谁高贵。」他蹲下来，在地上写写画画，「今天就用这个练判断逻辑——给你T组对局，每组两个人各出一拳，判断谁赢。相等就是平局。」

李少白眼睛一亮：「师兄，这不就是字符串比较吗？」

梁嘉峰站起身，拍拍裤腿上的土：「不错。rock、scissors、cloth——用字符串表示招式，用条件判断胜负。去练吧，记住：三局两胜不算本事，一局定输赢才见真章。」

输入格式

第一行整数T。接下来T行，每行两个字符串，表示两人出的拳。

输出格式

每行输出结果：Player1赢输出"Player1"，Player2赢输出"Player2"，平局输出"Tei"。

输入样例

```
scissors rock
```

输出样例

```
Player2
```

解题思路：

判断两者相等→平局；否则检查是否构成"石头>剪刀>布>石头"的环。

复杂度分析：

时间复杂度： $O(n)$ — 线性遍历处理

空间复杂度： $O(n)$

► 参考代码

JD073: 经文加密

简单 AcWing JD | JD073

深夜，赵晴儿在烛光下整理从密室取出的古卷。这些卷轴记载着门派的核心剑法，绝不能落入外人之手。

「少白，在存放之前，必须给每卷经文加上密锁。」她蘸了墨，在纸上写下规则，「最简单的加密之术——凯撒密码。每个字母往后推一位：'a'变成'b'，'b'变成'c'……到了'z'就绕回'a'。数字也一样，'9'变成'0'。至于空格、标点，原封不动。」

李少白接过笔，开始逐字替换。写了几个字后，他抬起头：「师姐，这也太慢了吧？一百个字要写一百遍。」

赵晴儿笑了：「所以才要写代码。遍历字符串，遇到字母就取模加一，遇到数字同理，其他字符跳过。一行循环，整卷加密。去写吧——这叫「经文加密」。」

输入格式

一行字符串（长度不超过100）。

输出格式

加密后的字符串。

输入样例

```
Hello, World!
```

输出样例

```
Ifmmp, Xpsme!
```

解题思路：

遍历字符串，对字母、数字分别做+1循环偏移，其他字符原样输出。

复杂度分析:

时间复杂度: $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

► [参考代码](#)

JD074：古卷替换

简单 AcWing JD | JD074

赵晴儿捧着一卷抄录本，眉头微蹙。李少白凑过去一看，原来抄录的人把「剑」字全写成了「刀」字，通篇读下来驴唇不对马嘴。

「少白，这卷经文被抄错了。有一个词A被误写成了另一个词B，全部都要改回来。」她把经文铺在桌上，用朱砂圈出每一处错误，「逐词核对，遇到A就替换成B，其他的保持不变。」

李少白拿起笔正要圈，赵晴儿递给他三张纸条：「第一张是原文，第二张是要找的词A，第三张是替换成的词B。在代码里，按空格分割单词，逐个比较，相等就替换。这叫「古卷替换」——字符串处理中最常用的操作之一。」

输入格式

三行：第一行原文字符串，第二行要替换的词A，第三行替换成的词B。

输出格式

替换后的字符串。

输入样例

```
first late system see school still great different year Place far take same  
life  
group
```

输出样例

```
first late system see school still great different year Place far take same
```

解题思路：

字符串替换函数，或按空格分割单词逐个比较替换。

复杂度分析:

时间复杂度: $O(n)$ — 线性遍历处理

空间复杂度: $O(n)$

► [参考代码](#)

JD075: 寻最长词

简单 AcWing JD | JD075

晨光透过竹帘洒在书案上，赵晴儿正在吟诵一卷古诗。李少白在一旁磨墨，忽然听到她停了下来。

「少白，古人行文讲究炼字——用最少的字表达最深的意。」她指着经文上的一句话，「但有时候，最长的那个词恰恰是关键所在。它可能是地名、人名，或者某种秘术的名称。」

李少白低头看去，句子以句号结尾，词与词之间用空格隔开。「要找出最长的那个词？」

「对。」赵晴儿放下经文，「按空格分割，逐个比较长度，记住最长的那一个。这叫「寻最长词」——看似简单，却是文本分析的基本功。记住，句子末尾有个句号，处理的时候别把它也算进词的长度里。」

输入格式

一行，以'.'结尾的英文句子（长度不超过500）。

输出格式

最长的单词。

输入样例

```
I am a student of Peking University.
```

输出样例

```
University
```

解题思路：

按空格分割单词（去掉末尾的'.'），遍历比较长度。

复杂度分析:

时间复杂度: $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

► [参考代码](#)

JD076：古卷插字

简单 AcWing JD | JD076

梁嘉峰从怀中掏出两片竹简残篇，小心翼翼地铺在石桌上。竹简已经发黄，边缘卷曲，上面的字迹却依然清晰。

「少白，这两片残篇本来是一卷完整的剑谱，被人撕成了两截。」他指着第一片，「这上面有一个字的内力最强——ASCII码最大的那个字符。找到它，然后把第二片残篇插到它的后面，剑谱就复原了。」

李少白拿起竹简翻看：「师兄，ASCII码最大的字符……就是遍历一遍，记住最大值的位置？」

「没错。」梁嘉峰靠在石桌旁，「找到最大字符的位置 p ，把第一片的前 $p+1$ 个字符、第二片全部、第一片剩下的部分拼在一起。这叫「古卷插字」——字符串拼接的基本操作。去写吧。」

输入格式

若干行，每行两个字符串 str 和 $substr$ ，用空格隔开。

输出格式

每行输出插入后的字符串。

输入样例

```
hello
X
```

输出样例

```
helloX
```

解题思路：

找到 str 中ASCII值最大的字符的位置，把 $substr$ 插在它后面。

复杂度分析:

时间复杂度: $O(n)$ — 线性遍历处理

空间复杂度: $O(n)$

► [参考代码](#)

JD077: 独字寻踪

简单 AcWing JD | JD077

藏经阁深处，赵晴儿举着灯笼照向一排排竹筒。火光映出密密麻麻的文字，有些字反复出现，如同重重叠叠的脚印。

「少白，这卷密文里有一个特殊的字——它只出现了唯一一次。」她把灯笼挂在竹架上，转过身来，「其他的字都成双成对、三五成群，唯独它，孤零零的，只露了一面。」

李少白仔细端详：「为什么要找它？」

赵晴儿微微一笑：「在密码学中，出现频率最低的字符往往是关键——它可能是某个数字的代号，或者某个机关的触发点。先统计每个字符出现了多少次，再从头遍历，找到第一个次数为一的。这叫「独字寻踪」。如果每个字都重复了，就输出"no"。」

输入格式

一行只包含小写字母的字符串。

输出格式

第一个只出现一次的字符，或"no"。

输入样例

```
abaccd eff
```

输出样例

```
b
```

解题思路：

先统计每个字符出现次数，再从头遍历找第一个次数为1的。

复杂度分析:

时间复杂度: $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

► [参考代码](#)

JD078: 两卷对校

简单 AcWing JD | JD078

赵晴儿拿着两卷古卷：「看看第二段经文是不是第一段中的一部分——如果是，就算对上了。」

输入格式

第一行一个整数 k （忽略，用于兼容）。第二行字符串 a 。第三行字符串 b （子串）。

输出格式

b 是 a 的子串输出"yes"，否则输出"no"。

输入样例

```
abc  
d
```

输出样例

```
no
```

解题思路：

遍历 a 的每个位置，用`strncmp`比较是否与 b 匹配。匹配则输出yes。`strncmp(a+i, b, len(b))==0`表示 b 是 a 的子串。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度： $O(1)$

► 参考代码

JD079: 经文周期

简单 AcWing JD | JD079

赵晴儿指着一篇重复的经文：「这段文字是某个子串重复几次写成的？找出最小的重复次数。」

输入格式

一行字符串。

输出格式

一行，最大的 n 。

输入样例

```
abcd  
aaaa  
ababab  
.
```

输出样例

```
1  
4  
3
```

解题思路：

枚举周期 p （ p 是 len 的因子），检查 $s[0:p]$ 重复 len/p 次是否等于原串。

复杂度分析：

时间复杂度： $O(n^2)$ — 枚举所有可能

空间复杂度： $O(1)$

▶ 参考代码

JD080: 密文跨距

简单 AcWing JD | JD080

夜深人静，梁嘉峰把李少白叫到密室。桌上摊着一长卷密文，旁边放着两枚短小的玉符——上面各刻着一个短码S1和S2。

「少白，这卷密文里藏着两个标记。S1是起始标记，S2是终结标记。」他用手指在密文上比划，「S1必须出现在S2的左边，两者不能交叉重叠。我需要知道——它们之间最大能隔多远？」

李少白思索片刻：「S1要尽量靠左，S2要尽量靠右？」

「对。」梁嘉峰点头，「从左往右找S1最后出现的位置，从右往左找S2最后出现的位置。如果S1的末尾在S2的开头之前，间距就是两者之间的距离。否则，输出-1——说明它们无法满足左S1右S2的条件。这叫「密文跨距」，是定位密文的关键技术。」

输入格式

一行，三个字符串用逗号隔开：S,S1,S2。

输出格式

最大跨距。不满足条件输出-1。

输入样例

```
abcd123dABdefghjsdef76ki,ab,ef
```

输出样例

```
16
```

解题思路：

从左往右找S1最后出现的位置end1，从右往左找S2最后出现的位置start2。如果end1

复杂度分析:

时间复杂度: $O(n)$ — 线性遍历处理

空间复杂度: $O(n)$

► [参考代码](#)

第7章 以招创招

函数与递归

丹房之中，赵晴儿演示炼丹之术："重复的操作，封装成函数。复杂的计算，用递归层层解开。"

本章学习函数封装和递归——将重复逻辑封装成可复用的函数，用递归解决分治类问题。这是从"写代码"到"设计程序"的飞跃。

JD081: 初窥阶乘

简单 AcWing JD | JD081

李少白在丹房帮赵晴儿炼丹。丹方上写着："取 n 味药材，第一味放1份，第二味放2份，第三味放3份……第 n 味放 n 份。"

赵晴儿问：" n 味药材总共需要多少份？"

李少白拿出笔从头算： $1 \times 2 \times 3 \times \dots \times n$ 。赵晴儿摇头："每次都要从头算，太慢了。把它封装成一个函数，以后直接调用。"

请帮李少白编写一个函数，计算 n 的阶乘 ($n!$)。

输入格式

一个整数 n ($0 \leq n \leq 10$)。

输出格式

输出 n 的阶乘。

输入样例

3

输出样例

6

解题思路：

函数内用循环累乘，或递归实现。注意 $0! = 1$ 。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

▶ [参考代码](#)

JD082：比剑取大

简单 AcWing JD | JD082

兵器库里，赵晴儿和李少白各自挑了一柄剑。赵晴儿的剑重 x 斤，李少白的剑重 y 斤。

赵晴儿说："我们总要比谁的剑更重，每次都手算太麻烦。写一个函数，传入两个数，返回较大的那个。以后比试直接调用就行。"

请帮他们实现这个函数。

输入格式

一行，两个整数 x 和 y 。

输出格式

输出最大值。

输入样例

3 5

输出样例

5

解题思路：

函数内用 *if* 比较或三元表达式。

复杂度分析：

时间复杂度： $O(n)$ — 线性遍历处理

空间复杂度： $O(n)$

▶ 参考代码

JD083: 剑气归正

简单 AcWing JD | JD083

李少白在练剑气外放时，需要计算剑气落点与自己的距离。距离本该是正数，但有时算出的结果是负数——因为方向搞反了。

赵晴儿说："别每次遇到负数就手动取反，写一个函数，不管传入正数还是负数，都返回它的绝对值。"

输入格式

一个整数 x 。

输出格式

输出 x 的绝对值。

输入样例

-5

输出样例

5

解题思路：

$\text{if } x < 0: \text{return } -x, \text{ else: return } x。$

复杂度分析：

时间复杂度： $O(n)$ — 线性遍历处理

空间复杂度： $O(n)$

▶ 参考代码

JD084: 双剑互易

简单 AcWing JD | JD084

梁嘉峰在教李少白一套双剑术。左右手各持一剑，攻守之间需要不断交换位置。

"实战中你不能把剑放到地上再捡起来，"梁嘉峰说，"直接交换。写一个函数，传入两个变量的引用，在函数内部交换它们的值。"

李少白提笔写下函数签名，却犯了难——C++里普通参数只传值，改不了外面的变量。

输入格式

一行，两个整数 x 和 y 。

输出格式

交换后的 x 和 y 。

输入样例

6 26

输出样例

26 6

解题思路：

C++用引用参数 `void swap(int &x, int &y)`。Python中 `a, b = b, a`。

复杂度分析：

时间复杂度： $O(n)$ — 线性遍历处理

空间复杂度： $O(n)$

▶ 参考代码

JD085: 辗转相除

简单 AcWing JD | JD085

赵晴儿在药圃里采了 a 株紫芝和 b 株青蒿，要把它们扎成相同大小的药束，每束株数相同且不能有剩余。

"这其实就是求 a 和 b 的最大公约数，"赵晴儿说，"古人有个妙法叫辗转相除——用大数除以小数取余，再用小数除以余数，反复如此，直到余数为零。最后一个非零余数就是答案。"

请帮赵晴儿写一个函数，用辗转相除法求两个正整数的最大公约数。

输入格式

一行，两个正整数 a 和 b 。

输出格式

输出 a 和 b 的最大公约数。

输入样例

963 787

输出样例

1

解题思路：

辗转相除法： $\gcd(a,b) = \gcd(b, a \% b)$ ，当 $b=0$ 时返回 a 。

复杂度分析：

时间复杂度： $O(\log(\min(a,b)))$ — 辗转相除法求GCD

空间复杂度： $O(1)$

▶ 参考代码

JD086: 剑阵复制

简单 AcWing JD | JD086

赵晴儿在沙盘上摆了一个剑阵（数组 a ），需要原样复制一份到另一个沙盘（数组 b ）上。

"手抄太慢，"赵晴儿说，"写一个函数，把 a 的前 $size$ 个元素复制到 b 对应的位置上。"

李少白接过两个沙盘，发现 b 上原来也有数字，只需覆盖前 $size$ 个位置，其余保持不变。

输入格式

第一行三个整数 n 、 m 和 $size$ （数组 a 长度、数组 b 长度、复制个数， $size \leq n$ 且 $size \leq m$ ）。

第二行 n 个整数，表示数组 a 。

第三行 m 个整数，表示数组 b 。

输出格式

复制后的数组 b ，空格隔开。

输入样例

```
3 5 2
1 2 3
4 5 6 7 8
```

输出样例

```
1 2 6 7 8
```

解题思路：

函数内循环， $b[i] = a[i]$ ，复制前 $size$ 个元素。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

▶ [参考代码](#)

JD087: 剑阵翻转

简单 AcWing JD | JD087

梁嘉峰在沙盘上摆了一排石块，每块上刻着一个数字。他指着前 $size$ 块说："剑阵需要反转——把第一个和最后一个交换，第二个和倒数第二个交换，依此类推。后面的石块不动。"

请帮李少白写一个函数，将数组的前 $size$ 个元素原地翻转。

输入格式

第一行两个整数 n 和 $size$ （数组长度和翻转个数， $size \leq n$ ）。

第二行 n 个整数，表示数组。

输出格式

翻转前 $size$ 个元素后的完整数组，空格隔开。

输入样例

```
5 3
1 2 3 4 5
```

输出样例

```
3 2 1 4 5
```

解题思路：

双指针从两端向中间交换，只交换前 $size$ 个。

复杂度分析：

时间复杂度： $O(n + m)$ — 双指针遍历

空间复杂度： $O(1)$

▶ 参考代码

JD088：印数成招

简单 AcWing JD | JD088

宗门兵器库盘点时，赵晴儿需要把清单上的前几件兵器名称逐一打印出来。清单上有 n 件兵器，但她只需要前 $size$ 件。

"每次都手动抄写太蠢了，"赵晴儿说，"写一个函数，传入数组和 $size$ ，把前 $size$ 个元素打印出来。"

输入格式

第一行两个整数 n 和 $size$ （清单总件数和待打印件数， $size \leq n$ ）。

第二行 n 个整数，表示清单。

输出格式

输出前 $size$ 个元素，空格隔开，末尾换行。

输入样例

```
5 3
1 2 3 4 5
```

输出样例

```
1 2 3
```

解题思路：

函数内遍历数组前 $size$ 个元素，空格隔开输出。

复杂度分析：

时间复杂度： $O(n)$ — 遍历数组

空间复杂度: $O(n)$

▶ [参考代码](#)

JD089: 印阵成招

简单 AcWing JD | JD089

赵晴儿在沙盘上画了一个row行col列的剑阵图，每个格子标着一个数字。她需要把这个阵图工整地抄录到竹简上。

"写一个函数，"赵晴儿说，"传入二维数组和行列数，按格式逐行打印。每行的数字用空格隔开，行末不要有多余空格。"

输入格式

第一行两个整数row和col。

接下来row行，每行col个整数。

输出格式

矩阵元素，每行一行，数字之间空格隔开，行末无多余空格。

输入样例

```
3 4
1 3 4 5
2 6 9 4
1 4 7 5
```

输出样例

```
1 3 4 5
2 6 9 4
1 4 7 5
```

解题思路：

嵌套循环遍历行列，注意行末空格处理。

复杂度分析:

时间复杂度: $O(n^2)$ — 嵌套循环

空间复杂度: $O(1)$

► [参考代码](#)

JD090: 剑阵排序

简单 AcWing JD | JD090

赵晴儿拿着一份兵器清单来找李少白："这份清单前半段乱得很，从第 l 件到第 r 件需要按重量从小到大排好，后面的顺序不变。"

李少白接过清单，发现上面有 n 件兵器的重量。他想写一个通用的排序函数，只排 $a[l]$ 到 $a[r]$ 这一段，其余位置原封不动。

输入格式

第一行三个整数 n 、 l 和 r （清单总件数、排序区间左端点和右端点， $0 \leq l \leq r < n$ ）。

第二行 n 个整数，表示清单上每件兵器的重量。

输出格式

排序后的完整清单，空格隔开。

输入样例

```
5 1 3
5 3 1 4 2
```

输出样例

```
5 1 3 4 2
```

解题思路：

对 $a[l]$ 到 $a[r]$ 排序，其余元素不变。可用冒泡、选择或库函数。

复杂度分析：

时间复杂度： $O(n \log n)$ — 排序算法

空间复杂度: $O(1)$

▶ [参考代码](#)

JD091: 递归阶乘

简单 AcWing JD | JD091

李少白在丹房翻到一本古籍，上面记载着一种"以丹炼丹"的奇术——要炼 n 味丹药，先炼好 $n-1$ 味；要炼 $n-1$ 味，先炼好 $n-2$ 味……直到只剩1味，直接炼成。

赵晴儿看了笑道："这就是递归。一个函数调用自己，层层深入，直到触底再层层返回。阶乘的定义本身就是递归的—— $\text{fact}(n) = n \times \text{fact}(n-1)$ ， $\text{fact}(1) = 1$ 。"

"用递归重新实现阶乘函数。"

输入格式

一个整数 n ($1 \leq n \leq 10$)。

输出格式

输出 n 的阶乘。

输入样例

5

输出样例

120

解题思路：

递归三要素：终止条件($n=1$ 返回1)、递推关系($n \times \text{fact}(n-1)$)、缩小规模。

复杂度分析：

时间复杂度： $O(n)$ — 递归 n 次，每次 $O(1)$

空间复杂度： $O(n)$

▶ 参考代码

JD092: 递归斐波

简单 AcWing JD | JD092

赵晴儿在沙盘上画出一串数字：1, 1, 2, 3, 5, 8, 13……

"看出规律了吗?"她问。

李少白看了半天："每个数都是前两个数的和！"

"对，这就是斐波那契数列。"赵晴儿说，"用递归来写—— $f(1)=1, f(2)=1, f(n)=f(n-1)+f(n-2)$ 。不过提醒你，朴素递归效率很低，后面会学到优化的方法。"

输入格式

一个整数 n ($1 \leq n \leq 20$)。

输出格式

输出斐波那契数列第 n 项。

输入样例

4

输出样例

3

解题思路：

递归： $f(1)=1, f(2)=1, f(n)=f(n-1)+f(n-2)$ 。注意效率问题。

复杂度分析：

时间复杂度： $O(2^n)$ — 递归，指数级时间复杂度

空间复杂度： $O(n)$

▶ 参考代码

JD093: 登阶问道

简单 AcWing JD | JD093

试炼场尽头，一座石阶直通山顶。石阶共有 n 级，每次只能跨1级或2级。

赵晴儿站在山脚问："从第一级走到第 n 级，一共有多少种不同的走法？"

李少白想了想："如果我最后一步跨1级，那之前有 $f(n-1)$ 种走法；如果最后一步跨2级，那之前有 $f(n-2)$ 种走法……"

"没错，"赵晴儿点头，"这就是递归。"

输入格式

一个整数 n ($1 \leq n \leq 20$)。

输出格式

输出走法总数。

输入样例

5

输出样例

8

解题思路：

$f(n) = f(n-1) + f(n-2)$ 。最后一步跳1级或2级，两种情况之和。

复杂度分析：

时间复杂度： $O(n)$ — 线性遍历处理

空间复杂度： $O(n)$

▶ 参考代码

JD094：方阵寻路

简单 AcWing JD | JD094

赵晴儿在沙盘上画了一个 $(n+1) \times (m+1)$ 的方格阵，每个格子标着坐标。

"假设你站在左上角 $(0,0)$ ，"赵晴儿指着沙盘说，"目标是走到右下角 (n,m) 。每次只能向右走一步，或者向下走一步。一共有多少种不同的路径？"

李少白盯着沙盘看了一会儿："走到 (n,m) 之前，一定是从 $(n-1,m)$ 或 $(n,m-1)$ 过来的……"

"很好，又是一个递归。"

输入格式

一行，两个整数 n 和 m （目标坐标， $0 \leq n, m \leq 20$ ）。

输出格式

输出路径总数。

输入样例

2 3

输出样例

10

解题思路：

从 $(0,0)$ 到 (n,m) 的网格路径数。 $f(n,m) = f(n-1,m) + f(n,m-1)$ 。边界： $f(0,m)=1, f(n,0)=1$ 。等价于组合数 $C(n+m, n)$ 。

复杂度分析：

时间复杂度： $O(n \times m)$ — DP计算网格路径，二维状态

空间复杂度: $O(n \times m)$

▶ [参考代码](#)

JD095：全排列阵

简单 AcWing JD | JD095

梁嘉峰递给李少白 n 枚令牌，上面分别刻着1到 n 。

"把它们排成一列，列出所有可能的排列顺序。"梁嘉峰说，"按字典序从小到大输出。"

李少白开始手动枚举，很快就乱了套。赵晴儿在旁边提醒："用递归的思路——每次选一个没用过的令牌放到当前位置，然后递归处理剩下的位置。选完之后要'回退'，把令牌放回去，才能尝试下一个选择。这就是回溯。"

输入格式

一个整数 n ($1 \leq n \leq 9$)。

输出格式

每行一个排列，数字之间用空格隔开。按字典序输出。

输入样例

3

输出样例

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

解题思路：

DFS+回溯：用`used[]`标记已用数字，`path[]`记录当前排列。

复杂度分析:

时间复杂度: $O(n!)$ — 回溯生成全排列, 共 $n!$ 种

空间复杂度: $O(n)$

► [参考代码](#)

第8章 百器图谱

容器与内置算法

兵器阁深处，梁嘉峰指着各种容器："栈如叠盘，队如列阵，链如铁环——每种容器都有独特的用途。"

本章学习 STL 容器——栈、队列、链表等数据结构的实际应用。掌握容器，你的代码将更加优雅高效。

JD096：空格替换

简单 AcWing JD | JD096

兵器阁的大门上刻着一行古文，但年代久远，字迹模糊，空格处尤其难以辨认。赵晴儿凑近看了看："这行字里夹了太多空格，密密麻麻的，根本看不清。"

梁嘉峰说："古卷传输时，空格容易丢失。古人想了个办法——把每个空格替换成 `%20`，这样既保留了间隔，又不会和真正的空白混淆。"

"你写一段程序，把这行古文里的所有空格替换成 `%20`。"

输入格式

一行字符串（长度不超过100），可能包含空格。

输出格式

替换空格为 `%20` 后的字符串。

输入样例

```
Hello World
```

输出样例

```
Hello%20World
```

解题思路：

遍历字符串，遇到空格输出 `%20`，否则输出原字符。可用 `string` 的 `getline` 读入含空格的整行。

复杂度分析：

时间复杂度： $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

▶ [参考代码](#)

JD097：再算斐波

简单 AcWing JD | JD097

赵晴儿在兵器阁的账本上翻到一串奇怪的数字：1, 1, 2, 3, 5, 8, 13, 21……

"这是前辈铸剑师留下的排列规律，"梁嘉峰说，"每一柄新剑的重量都是前两柄之和。第1柄和第2柄都是1斤，第3柄就是 $1+1=2$ 斤，第4柄是 $1+2=3$ 斤……"

"上一章你用递归算过这个数列，但递归太慢了。这次用容器来存——把每一项都推进一个序列里，只保留最近两项，循环推进。"

请帮赵晴儿算出第 n 柄剑的重量。

输入格式

一个整数 n ($0 \leq n \leq 90$)。

输出格式

输出斐波那契数列第 n 项。约定第0项为0，第1项为1。

输入样例

10

输出样例

55

解题思路：

迭代法：用两个变量 `a` 和 `b` 滚动，每次 `c = a + b`，然后 `a = b`, `b = c`。循环 n 次。也可以用 `vector` 存储每一项。

复杂度分析:

时间复杂度: $O(n)$ — 单层循环遍历

空间复杂度: $O(1)$

► [参考代码](#)

JD098：倒转铁链

简单 AcWing JD | JD098

兵器阁的架子上挂着一串铁环，每个环上刻着一个数字，从头到尾依次是 1、2、3……最后一个环上刻着 -1，表示链条结束。

梁嘉峰指着铁链说："我要从最后一个环开始，把数字一个个报出来。但这串铁环只能从前往后读，不能直接跳到末尾。"

赵晴儿想了想："用一个容器——先把所有数字存进去，再从后往前取出来。这就像把铁环一个个摘下来放到盒子里，最后从盒子里倒着取。"

输入格式

一行整数，以 -1 结尾，表示铁链上每个环的数字（-1 不算节点）。

输出格式

从尾到头输出每个数字，每行一个。

输入样例

```
1 2 3 -1
```

输出样例

```
3
2
1
```

解题思路：

用 `vector` 依次存入所有数字，然后从后往前遍历输出。也可以用 `stack`：读入时压栈，读完后依次弹出。

复杂度分析:

时间复杂度: $O(n)$ — 栈操作, 每个元素最多入栈出栈各一次

空间复杂度: $O(n)$

► [参考代码](#)

JD099: 双栈为队

简单 AcWing JD | JD099

赵晴儿在兵器阁里找到两个铁盒，每个盒子只能从顶部放入和取出——这就是"栈"，后进先出。

梁嘉峰说："现在我需要一个'队列'——先进先出。但手边只有这两个栈，没有队列。你能用这两个栈模拟一个队列吗？"

赵晴儿思索片刻："入队的时候都放进盒子1。出队的时候，如果盒子2是空的，就把盒子1的东西全部倒进盒子2——这样顺序就反过来了，最先进来的变成了盒子2的顶部，直接取就行。"

请帮赵晴儿实现这个"双栈模拟队列"的操作。

输入格式

若干行操作命令：

- `push x`：将 `x` 入队
- `pop`：出队并输出队首值
- `empty`：判空，输出 `yes` 或 `no`

输出格式

对每个 `pop` 操作输出一行队首值，对每个 `empty` 操作输出一行 `yes` 或 `no`。

输入样例

```
push 1
push 2
pop
push 3
pop
pop
empty
```

输出样例

```
1
2
3
yes
```

解题思路：

用两个 `stack`：栈1负责入队，栈2负责出队。出队时若栈2为空，把栈1全部倒入栈2（`push` 栈1的 `top` 到栈2，再 `pop` 栈1）。这样栈2的顶部就是最早入队的元素。

复杂度分析：

时间复杂度： $O(n)$ — 栈操作，每个元素最多入栈出栈各一次

空间复杂度： $O(n)$

► 参考代码

JD100：铁链翻转

简单 AcWing JD | JD100

梁嘉峰从架子上取下一条铁链，每个环上刻着一个数字。他把铁链平铺在桌上，从头到尾是 1、2、3、4、5……最后一个环上刻着 -1，表示结束。

"我要把这条铁链反过来，"梁嘉峰说，"第一个变最后一个，最后一个变第一个。但不能断开重接，只能用容器来辅助。"

赵晴儿拿起一个铁盒："把铁环一个个摘下来放进盒子里——先进去的沉到底部。等全部摘完，再从盒子里取出来，顺序就反了。"

输入格式

一行整数，以 -1 结尾，表示铁链上每个环的数字（-1 不算节点）。

输出格式

反转后的铁链，数字用空格隔开。

输入样例

```
1 2 3 4 5 -1
```

输出样例

```
5 4 3 2 1
```

解题思路：

用 `vector` 读入所有数字，然后从后往前输出。或者用 `stack`：读入时压栈，读完后依次弹出。栈的后进先出特性天然实现了反转。

复杂度分析:

时间复杂度: $O(n)$ — 栈操作, 每个元素最多入栈出栈各一次

空间复杂度: $O(n)$

► [参考代码](#)

JD101: 双链归并

简单 AcWing JD | JD101

赵晴儿从兵器阁的两个抽屉里各取出一条铁链，每条链上的数字都从小到大排好了。她需要把两条链合并成一条，仍然保持从小到大的顺序。

"不能拆开重排，"梁嘉峰说，"两条链都是有序的，你要利用这个特点——从两条链的头部同时开始比较，每次取较小的那个接到结果链的末尾。"

赵晴儿拿起两个容器："用一个容器存结果。两个指针分别指向两条链的头部，谁小就取谁，直到一条链取完，再把另一条的剩余全部接上。"

输入格式

两行，每行若干整数（以 -1 结尾），分别表示两条有序铁链上的数字（-1 不算节点）。

输出格式

合并后的有序铁链，数字用空格隔开。

输入样例

```
1 3 5 -1
2 4 6 -1
```

输出样例

```
1 2 3 4 5 6
```

解题思路：

用 `vector` 分别读入两条链，然后双指针合并：比较两个 `vector` 当前元素，取较小的放入结果 `vector`。一个遍历完后，把另一个的剩余部分直接追加。最后用 `sort` 也可，但双指针更高效。

复杂度分析:

时间复杂度: $O(n + m)$ — 双指针遍历

空间复杂度: $O(1)$

► [参考代码](#)

JD102: 三元归序

简单 AcWing JD | JD102

兵器阁的账房先生有一本厚厚的兵器账本，每条记录记着三样东西：兵器编号 x 、重量 y （带两位小数）、兵器名称 z 。

"这本账太乱了，"赵晴儿翻了几页就头疼，"编号不按顺序，同编号的重量也乱排。"

梁嘉峰说："先按编号从小到大排，编号相同的再按重量从小到大排。你可以用结构体存每条记录，放进容器里，再自定义排序规则。"

输入格式

第一行一个整数 N （记录条数）。

接下来 N 行，每行两个整数和一个字符串，分别为编号 x 、重量 y 、名称 z 。

输出格式

排序后的 N 条记录，每行一条。编号输出整数，重量保留两位小数，名称原样输出，空格隔开。

输入样例

```
3
1 2 abc
3 4 def
2 1 ghi
```

输出样例

```
1 2.00 abc
2 1.00 ghi
3 4.00 def
```

解题思路：

用 `struct` 定义三元组（编号、重量、名称），存入 `vector`。自定义比较函数：先比编号，编号相同再比重量。用 `sort(a.begin(), a.end(), cmp)` 排序。

复杂度分析：

时间复杂度： $O(n \log n)$ — 排序算法

空间复杂度： $O(1)$

► 参考代码

第9章 快剑如风

排序与二分

试炼场上，梁嘉峰演示一剑分两路之术："排序——让混乱变得有序。二分——在有序中快速定位。"

本章学习快速排序、归并排序和二分查找——这是算法世界的基石，掌握它们，你将拥有处理海量数据的能力。

JD103: 立桩定阵

简单 AcWing 785 | JD103

试炼场上，弟子们按身高排成一列，但顺序完全乱了。梁嘉峰走过去，随便挑了一个弟子站到中间，然后让所有比他矮的站到左边，比他高的站到右边。

"现在左边和右边各自还是乱的，"赵晴儿看出来了。

"对。所以对左边和右边各自重复这个过程——挑一个人，矮的左、高的右。每次都能让一个人站到最终位置上，直到每一组只剩一个人。"梁嘉峰说，"这就是快速排序。一剑分两路，递归再出剑。"

给定一个长度为 n 的整数数列，请用快速排序将其从小到大排序。

输入格式

第一行包含整数 n 。第二行包含 n 个整数。

输出格式

一行， n 个排好序的整数，空格隔开。

输入样例

```
5
3 1 2 4 5
```

输出样例

```
1 2 3 4 5
```

解题思路：

选基准元素，双指针分区：左边 $\leq$ 基准，右边 $>$ 基准。递归处理左右两半。平均 $O(n\log n)$ 。

复杂度分析:

时间复杂度: $O(n \log n)$ — 快速排序, 平均每次分治 $O(n)$, 递归深度 $O(\log n)$

空间复杂度: $O(\log n)$

► [参考代码](#)

JD104: 照妖辨品

简单 AcWing 786 | JD104

梁嘉峰指着那排弟子："我不需要所有人都排好——只要知道第 K 矮的人是谁。"

赵晴儿想了想："还是用刚才的方法：挑一个人站中间，矮的左、高的右。数一数左边有几个人——如果左边刚好有 $K-1$ 个人，那中间那个就是答案。如果左边人比 $K-1$ 多，说明答案在左边，只管左边就行。如果不够，就去右边找第 $K-左-1$ 矮的。"

"所以快排是把两边都排好，快选只需要管一边。"梁嘉峰点头，"省了一半的力气。"

给定一个长度为 n 的整数数列和一个整数 k ，求数列中第 k 小的数。

输入格式

第一行两个整数 n 和 k 。第二行 n 个整数。

输出格式

一行，第 k 小的数。

输入样例

```
5 3
3 1 2 4 5
```

输出样例

```
3
```

解题思路：

`partition` 后判断 k 在左半还是右半，只递归一半。平均 $O(n)$ 。

复杂度分析:

时间复杂度: $O(n)$ — 快速选择, 平均每次只递归一半

空间复杂度: $O(n)$

► [参考代码](#)

JD105: 双剑合阵

简单 AcWing 787 | JD105

赵晴儿把弟子们两两分成一组，每组内部先按身高排好。然后两组合成一组四人——从两组的排头各看一眼，矮的先出列，依次接上。四人组再两两合并成八人组……

梁嘉峰看明白了："快排是先挑基准把人分开，归并是先把人拆到最小单位，再一层层合并上来。"

"对，"赵晴儿说，"合并时两边都是有序的，所以每次只需要比较排头， $O(n)$ 就能合并。拆到底再合上来，总共 $O(n\log n)$ 。而且最坏情况也一样——不像快排最坏会退化。"

给定一个长度为 n 的整数数列，请用归并排序将其从小到大排序。

输入格式

第一行包含整数 n 。第二行包含 n 个整数。

输出格式

一行， n 个排好序的整数，空格隔开。

输入样例

```
5
3 1 2 4 5
```

输出样例

```
1 2 3 4 5
```

解题思路：

递归拆分 → 合并两个有序数组。需要额外空间 $O(n)$ 。稳定排序。

复杂度分析:

时间复杂度: $O(n \log n)$ — 归并排序, 分治递归深度 $O(\log n)$, 每层合并 $O(n)$

空间复杂度: $O(n)$

► [参考代码](#)

JD106：逆流之数

简单 AcWing 788 | JD106

弟子们排成一列，每人身上有一个编号。赵晴儿发现，有些排在前面的人编号反而比后面的大——这就是"逆序对"，说明队伍很乱。

"逆序对越多，队伍越乱。"赵晴儿说，"但要一个个比太慢了。有没有 $O(n\log n)$ 的方法？"

李少白盯着归并排序看了一会儿，突然开窍："归并的时候，左右两队都是从小到大排好的。从两队的排头开始比较——如果右边的先出列，说明右边这个人比左边队里剩下的所有人都小，那些都是逆序对！"

"没错。"赵晴儿笑了，"排完序，逆序对也数完了。"

给定一个长度为 n 的整数数列，求逆序对的数量。逆序对定义： $i < j$ 且 $a[i] > a[j]$ 。

输入格式

第一行包含整数 n 。第二行包含 n 个整数 ($1 \leq n \leq 100000$)。

输出格式

一行，逆序对的数量。

输入样例

```
5
2 3 1 5 4
```

输出样例

```
3
```

解题思路：

归并排序过程中，当右边元素先被选中时，左边剩余元素个数就是新增的逆序对数。用

`long long` 存结果。

复杂度分析：

时间复杂度： $O(n \log n)$ — 归并排序

空间复杂度： $O(n)$

► 参考代码

JD107: 石壁听风

简单 AcWing 789 | JD107

赵晴儿递给李少白一排刻着数字的石块，从左到右从小到大排好了。她问："数字 3 第一次出现在哪块石块上？最后一次出现在哪？"

李少白想从头数，但石块有十万块。赵晴儿拦住他："石块是有序的，不用一个个看。先看最中间那块——如果中间的数比 3 大，说明 3 只可能在左半边，右半边不用看了。每次都把范围砍一半，这就是二分。"

"找第一次出现的位置和最后一次出现的位置，各二分一次。"李少白接话道。

"对。如果整个范围里都找不到，就说明没有。"

给定一个升序排列的长度为 n 的整数数组和 q 个查询，对每个查询返回目标值的起始位置和终止位置（从 0 开始），不存在则返回 `-1 -1`。

输入格式

第一行两个整数 n 和 q 。第二行 n 个整数（升序）。第三行 q 个待查询的整数。

输出格式

每行两个整数，起始位置和终止位置。

输入样例

```
6 3
1 2 2 3 3 4
2 3 5
```

输出样例

```
1 2
3 4
-1 -1
```

解题思路：

两次二分：找第一个 $\geq x$ 的位置（左边界），找最后一个 $\leq x$ 的位置（右边界）。

复杂度分析：

时间复杂度： $O(\log n)$ — 二分查找

空间复杂度： $O(1)$

► **参考代码**

JD108: 剑指方根

简单 AcWing 790 | JD108

赵晴儿在沙盘上写下一个数 n : "求它的三次方根。"

李少白试了几个整数, 立方后不是太大就是太小。赵晴儿说: "别猜了。三次方根一定在 -10000 到 10000 之间。你取中间值试试——如果中间值的立方比 n 大, 说明真正的答案在左半边; 比 n 小, 就在右半边。范围不断缩小, 直到精度足够。"

"和整数二分一样的思路," 李少白说, "只不过这次找的不是一个确切的位置, 而是一个足够精确的浮点数。"

"对。整数二分停在某个整数上, 浮点二分停在精度达标时。"

给定一个浮点数 n , 求它的三次方根。

输入格式

一个浮点数 n ($-10000 \leq n \leq 10000$)。

输出格式

n 的三次方根, 保留6位小数。

输入样例

1000.000000

输出样例

10.000000

解题思路:

浮点二分: $left = -10000, right = 10000, while(right - left > 1e-8), mid = (left + right) / 2$ 。若 $mid^3 \geq n$, $right = mid$; 否则 $left = mid$ 。

复杂度分析:

时间复杂度: $O(\log n)$ — 二分查找

空间复杂度: $O(1)$

► [参考代码](#)

第10章 千锤百炼

高精度与位运算

精铸阁中，匠人在铁板上刻下巨大的数字。“高精度——用数组模拟大数运算。位运算——在二进制世界中游刃有余。”

本章学习高精度运算和位运算——当普通数据类型不够用时，这些技巧能让你突破极限。

JD109: 铁壁识痕

简单 AcWing 801 | JD109

精铸阁的墙壁上嵌着一排铁板，每块铁板上用二进制刻着一个数——有凹痕的地方是1，平整的地方是0。

赵晴儿递给李少白一串数字："每块铁板上有几道凹痕？数出来。"

李少白想逐位检查。赵晴儿教他一个窍门："用位运算——`n & 1` 看最低位是不是1，然后 `n >>= 1` 右移一位，继续看下一位。数完所有位，就知道有几道凹痕了。"

给定 n 个整数，输出每个数的二进制表示中有多少个1。

输入格式

第一行一个整数 n 。第二行 n 个整数 ($1 \leq n \leq 100000$)。

输出格式

一行，每个数对应的1的个数，空格隔开。

输入样例

```
5
1 2 3 4 5
```

输出样例

```
1 1 2 1 2
```

解题思路：

位运算：`while(n > 0) { count += n & 1; n >>= 1; }`。或用 `n & (n-1)` 消去最低位的1，计数更快。

复杂度分析:

时间复杂度: $O(\log n)$ — 位运算, 逐位处理

空间复杂度: $O(1)$

► [参考代码](#)

JD110: 万钧合一

简单 AcWing 791 | JD110

精铸阁中，赵晴儿递来两块铁牌，上面各刻着一个巨大的数字——长到普通变量根本装不下。

"这两个数要加起来，"赵晴儿说，"但没有那么大的容器。怎么办？"

李少白想了想："像竖式加法一样，把每一位拆开，用数组存。从最低位开始逐位相加，满十进一。"

"对。这就是高精度加法——用数组模拟竖式。"

给定两个正整数，计算它们的和。每个数不超过 100000 位。

输入格式

两行，每行一个正整数。

输出格式

一行，两数之和。

输入样例

```
123456789012345678901234567890
987654321098765432109876543210
```

输出样例

```
111111111011111111101111111100
```

解题思路：

用数组存每一位（倒序），从低位到高位逐位相加，处理进位。最后倒序输出。

复杂度分析:

时间复杂度: $O(n)$ — 高精度加法, 逐位处理进位

空间复杂度: $O(n)$

► [参考代码](#)

JD111: 削铁如泥

简单 AcWing 792 | JD111

梁嘉峰递给李少白两块铁牌："这次是减法。大数减大数，注意借位。"

李少白接过一看，两个数都很长。他先比较了大小——如果被减数比减数小，结果就是负数，先交换再算。

"像竖式减法一样，从最低位开始，不够减就向高位借1。"梁嘉峰说，"削铁如泥，一刀一刀来。"

给定两个正整数，计算它们的差（结果可能为负数）。每个数不超过 100000 位。

输入格式

两行，每行一个正整数。

输出格式

一行，两数之差。

输入样例

```
9876543210
1234567890
```

输出样例

```
8641975320
```

解题思路：

先判断大小，大减小，从低位逐位减，处理借位。如果被减数小则结果加负号。

复杂度分析:

时间复杂度: $O(n)$ — 高精度减法, 逐位处理借位

空间复杂度: $O(n)$

► [参考代码](#)

JD112: 叠甲千层

简单 AcWing 793 | JD112

赵晴儿递来一块刻着万位数的铁牌和一个小铁块："大数乘以小数。铁牌上的每一位都要乘以小铁块的数，再处理进位——就像叠甲，一层压一层。"

李少白从铁牌的最低位开始，逐位乘以小数，加上前一位的进位，本位留下余数，进位带到下一位。

给定一个大正整数 A 和一个小正整数 B ，计算 $A \times B$ 。 A 不超过 100000 位， B 不超过 10000。

输入格式

两行，第一行正整数 A ，第二行正整数 B 。

输出格式

一行， $A \times B$ 的结果。

输入样例

```
123456
789
```

输出样例

```
97406784
```

解题思路：

高精度 $\times$ 低精度：大数每一位乘以小数，处理进位。注意结果可能有前导零需要去掉。

复杂度分析：

时间复杂度： $O(n)$ — 高精度乘低精度，逐位处理

空间复杂度: $O(n)$

▶ [参考代码](#)

JD113：分金断玉

简单 AcWing 794 | JD113

梁嘉峰拿起一把锉刀："大数除以小数——分金断玉，从高位一刀一刀切。"

"从最高位开始，"赵晴儿解释，"当前余数乘10加上当前位，除以除数得到商的一位，剩下的余数带到下一位。和竖式除法一模一样。"

给定一个大非负整数 A 和一个小正整数 B ，计算 $A \div B$ 的商和余数。 A 不超过 100000 位， B 不超过 10000。

输入格式

两行，第一行非负整数 A ，第二行正整数 B ($B \neq 0$)。

输出格式

两行，第一行商，第二行余数。

输入样例

```
9876543210
123
```

输出样例

```
80297099
33
```

解题思路：

高精度 $\div$ 低精度：从高位到低位，当前余数 $\times 10 +$ 当前位，除以 B 得商的一位，更新余数。

复杂度分析：

时间复杂度： $O(n)$ — 高精度除低精度，逐位处理

空间复杂度: $O(n)$

▶ [参考代码](#)

JD114：双剑求和

简单 AcWing 800 | JD114

赵晴儿递给李少白两排有序的铁牌，每排上的数字从小到大排列。她说："从两排铁牌中各取一块，使它们的数字之和刚好等于一个目标值。"

李少白想暴力枚举，但每排都有十万块。赵晴儿说："两排都是有序的——第一个指针从第一排的头部开始，第二个指针从第二排的尾部开始。如果和太大，第二个指针往左移；和太小，第一个指针往右移。两个指针配合，一次遍历就够了。"

给定两个升序数组 A 和 B ，以及目标值 x ，找出满足 $A[i] + B[j] = x$ 的数对 (i, j) 。保证有唯一解。

输入格式

第一行三个整数 n 、 m 和 x 。第二行 n 个整数（数组 A ）。第三行 m 个整数（数组 B ）。

输出格式

一行，两个整数 i 和 j 。

输入样例

```
4 5 6
1 2 4 7
3 4 6 8 9
```

输出样例

```
1 1
```

解题思路：

双指针： i 从 A 的头， j 从 B 的尾。 $A[i]+B[j] < x$ 则 $i++$ ， $> x$ 则 $j--$ 。 $O(n+m)$ 。

复杂度分析:

时间复杂度: $O(n + m)$ — 双指针遍历

空间复杂度: $O(1)$

► [参考代码](#)

JD115: 双剑验序

简单 AcWing 2816 | JD115

赵晴儿拿着两卷竹简，上面各写着一串数字。她问："第二卷的内容是不是第一卷的子序列？子序列不要求连续，但必须保持原有次序。"

李少白用两个指针："一个指针遍历第一卷，另一个指向第二卷。如果当前字符匹配，第二卷的指针就往前走。最后看第二卷的指针有没有走到头——走到头就说明全部匹配上了。"

"对，两个指针各走一遍， $O(n)$ 。"

给定序列 a （长度 n ）和序列 b （长度 m ），判断 a 是否为 b 的子序列。

输入格式

第一行两个整数 n 和 m 。第二行 n 个整数（序列 a ）。第三行 m 个整数（序列 b ）（ $1 \leq n \leq m \leq 100000$ ）。

输出格式

是子序列输出 **Yes**，否则输出 **No**。

输入样例

```
3 5
1 3 5
1 2 3 4 5
```

输出样例

Yes

解题思路：

双指针： i 遍历 a ， j 遍历 b 。 $a[i] == b[j]$ 时 $i++$ 。 i 到头说明是子序列。

复杂度分析:

时间复杂度: $O(n + m)$ — 双指针遍历

空间复杂度: $O(1)$

► [参考代码](#)

JD116：无重之最

简单 AcWing 799 | JD116

梁嘉峰在沙盘上摆了一排石块，每块上刻着一个数字。他问："从这排石块中，找出最长的一段——这段里每个数字都不重复。"

李少白想从头遍历每一段，但石块有十万块。赵晴儿教他："用两个指针，一左一右，框住一段区间。右指针往右扩展，遇到重复数字就收缩左指针，直到区间内没有重复。每一步都更新最大长度。"

"两个指针各走各的，总共只走一遍—— $O(n)$ 。"

给定一个长度为 n 的整数序列，找出最长的不含重复数字的连续子序列，输出其长度。

输入格式

第一行一个整数 n 。第二行 n 个整数 ($1 \leq n \leq 100000$)。

输出格式

一行，最长连续不重复子序列的长度。

输入样例

```
5
1 2 2 3 5
```

输出样例

```
3
```

解题思路：

滑动窗口：右指针扩展，用哈希表判断是否重复，重复则左指针收缩。 $O(n)$ 。

复杂度分析:

时间复杂度: $O(n)$ — 哈希表查找, 平均 $O(1)$

空间复杂度: $O(n)$

► [参考代码](#)

第11章 蓄势待发

前缀和、差分与双指针

蓄势阁中，赵晴儿盘膝而坐："前缀和——记住历史，快速回答。差分——记录变化，高效更新。双指针——两路并进，一击即中。"

本章学习前缀和、差分和双指针——这些技巧让你用 $O(1)$ 或 $O(n)$ 的时间处理看似需要 $O(n^2)$ 的问题。

JD121: 蓄势之术

简单 AcWing 795 | JD121

蓄势阁中，赵晴儿盘膝而坐，面前摆着一排石块，每块上刻着一个数字。

"如果我问你第3块到第7块的总和，你怎么算？"赵晴儿问。

李少白："从第3块加到第7块，五次加法。"

"如果问一千次呢？"赵晴儿摇头，"每次从头算太浪费了。提前算好从第1块到每一块的累积和——第1块到第7块的和减去第1块到第2块的和，就是第3块到第7块的和。一次减法就够了。"

"这就是前缀和——蓄势于前，取用于后。"

给定一个长度为 n 的整数序列和 m 个查询，每个查询求 $[l, r]$ 的区间和。

输入格式

第一行两个整数 n 和 m 。第二行 n 个整数。接下来 m 行，每行两个整数 l 和 r （从1开始计数）。

输出格式

m 行，每行一个整数。

输入样例

```
5 3
2 1 3 6 4
1 3
2 4
1 5
```

输出样例

```
6
10
16
```

✦ **解题思路：**

前缀和： $s[i] = a[1] + a[2] + \dots + a[i]$ 。区间和 = $s[r] - s[l-1]$ 。

复杂度分析：

时间复杂度： $O(n)$ — 前缀和预处理， $O(1)$ 查询

空间复杂度： $O(n)$

► **参考代码**

JD122: 方阵蓄势

简单 AcWing 796 | JD122

赵晴儿在沙盘上画了一个 n 行 m 列的数字矩阵。"如果我问你左上角 $(x1,y1)$ 到右下角 $(x2,y2)$ 这个子矩阵里所有数的和，你怎么算？"

李少白想逐行累加。赵晴儿摇头："太慢了。和一维前缀和一样的思路——提前算好从 $(1,1)$ 到每个位置的累积和。查询时用容斥原理：大矩形减去左边和上边的多余部分，加上左上角多减的部分。"

"蓄势蓄到二维，一击覆盖任意子矩阵。"

给定一个 $n \times m$ 的整数矩阵和 q 个查询，每个查询求子矩阵的和。

输入格式

第一行三个整数 n 、 m 和 q 。接下来 n 行，每行 m 个整数。接下来 q 行，每行四个整数 $x1$ 、 $y1$ 、 $x2$ 、 $y2$ 。

输出格式

q 行，每行一个整数。

输入样例

```
3 4 2
1 3 4 5
2 6 9 4
1 4 7 5
1 1 2 2
2 1 3 3
```

输出样例

```
18
43
```

解题思路：

二维前缀和： $s[i][j]$ = 左上角到 (i,j) 的累加。子矩阵和 = $s[x2][y2] - s[x1-1][y2] - s[x2][y1-1] + s[x1-1][y1-1]$ 。

复杂度分析：

时间复杂度： $O(n^2)$ — 二维前缀和

空间复杂度： $O(n^2)$

► 参考代码

JD123: 差分之术

简单 AcWing 797 | JD123

赵晴儿指着一排石块："我要把第3块到第7块每块都加上5。如果一块块改，太慢了。"

梁嘉峰说："差分——前缀和的逆运算。你在第3块的位置加5，在第8块的位置减5。最后对整排求一次前缀和，第3到第7块就都加了5，其余不变。"

"就像往水面扔两块石头，"李少白恍然大悟，"波纹叠加后就是最终结果。"

给定一个长度为 n 的整数序列和 m 个操作，每个操作将 $[l, r]$ 的每个数加 c 。输出所有操作后的序列。

输入格式

第一行两个整数 n 和 m 。第二行 n 个整数。接下来 m 行，每行三个整数 l 、 r 和 c 。

输出格式

一行，操作后的序列，空格隔开。

输入样例

```
5 3
1 2 3 4 5
1 3 2
2 4 1
3 5 3
```

输出样例

```
3 5 8 8 8
```

解题思路：

差分： $b[l] += c$, $b[r+1] -= c$ 。所有操作完成后，对 b 求前缀和得到最终序列。

复杂度分析:

时间复杂度: $O(n)$ — 前缀和预处理, $O(1)$ 查询

空间复杂度: $O(n)$

► [参考代码](#)

JD124: 方阵差分

简单 AcWing 798 | JD124

赵晴儿在沙盘上画了一个 n 行 m 列的矩阵："我要把左上角 $(x1,y1)$ 到右下角 $(x2,y2)$ 这个子矩阵里每个数都加上 c 。"

梁嘉峰说："和一维差分一样的思路——在差分矩阵的四个角加减，最后对整个矩阵求一次二维前缀和，就还原出最终结果了。"

"蓄势蓄到二维，差分也能覆盖任意子矩阵。"

给定一个 $n \times m$ 的整数矩阵和 q 个操作，每个操作将子矩阵 $(x1,y1)-(x2,y2)$ 的每个元素加 c 。输出所有操作后的矩阵。

输入格式

第一行三个整数 n 、 m 和 q 。接下来 n 行，每行 m 个整数。接下来 q 行，每行五个整数 $x1$ 、 $y1$ 、 $x2$ 、 $y2$ 和 c 。

输出格式

n 行，每行 m 个整数，空格隔开。

输入样例

```
3 4 2
1 2 3 4
5 6 7 8
9 10 11 12
1 1 2 2 1
2 2 3 3 2
```

输出样例

```
2 3 3 4
6 10 10 8
9 12 14 12
```

✦ **解题思路：**

二维差分：在 $(x1,y1)+c$, $(x2+1,y1)-c$, $(x1,y2+1)-c$, $(x2+1,y2+1)+c$ 。操作完后求二维前缀和。

复杂度分析：

时间复杂度： $O(n^2)$ — 二维前缀和

空间复杂度： $O(n^2)$

► **参考代码**

JD125: 离散聚力

简单 AcWing 802 | JD125

赵晴儿指着一条无限长的数轴："坐标从负无穷到正无穷，但真正有值的位置只有几个。如果开一个那么大的数组，太浪费了。"

"所以要把这些稀疏的大坐标映射到连续的小下标上——这就是离散化。"梁嘉峰接话，"映射之后再使用前缀和，就能 $O(1)$ 回答区间求和的查询。"

"先聚力于有用之处，再一击命中。"

假定有一个无限长的数轴，初始每个坐标上都是0。先进行 n 次操作，每次将位置 x 上的数加 c 。然后进行 m 次查询，每次求区间 $[l, r]$ 的和。

输入格式

第一行两个整数 n 和 m 。

接下来 n 行，每行两个整数 x 和 c 。

接下来 m 行，每行两个整数 l 和 r 。

输出格式

m 行，每行一个整数。

输入样例

```
3 3
1 2
3 6
7 5
1 3
4 6
7 8
```

输出样例

```
8
0
```

解题思路：

离散化的核心步骤：

1. **收集坐标：**把所有用到的坐标（加法操作的位置 x 、查询区间的端点 l 和 r ）全部收集到一个数组 `alls` 中。
2. **排序去重：**对 `alls` 排序并去重，得到一个从小到大排列的"映射表"。例如 `alls = [1, 3, 7]`。
3. **建立映射：**用二分查找（或哈希表）将原始坐标映射到连续下标： $1 \rightarrow 1, 3 \rightarrow 2, 7 \rightarrow 3$ 。这样原来稀疏的坐标就被"压缩"到了紧凑的数组下标上。
4. **前缀和：**在映射后的数组上做前缀和，就能 $O(1)$ 回答区间查询。

离散化的本质：坐标值可能很大（如 10^9 ），但实际用到的坐标只有少数几个。把这些稀疏坐标映射到连续小下标，既节省空间，又能用前缀和等技巧高效处理。

复杂度分析：

时间复杂度： $O(n \log n)$ — 排序算法

空间复杂度： $O(1)$

► 参考代码

JD126: 合围归阵

简单 AcWing 803 | JD126

梁嘉峰在地上画了一堆区间，有些重叠，有些相邻。"把这些区间合并——有交集的归为一组，最后数一数有几组。"

赵晴儿蹲下来看了一会儿："先按左端点从小到大排。然后从左往右扫——如果当前区间的左端点在前一个区间内，说明它们重叠，合并；否则新开一组。"

"合围之势，聚散有序。"

给定 n 个区间 $[l_i, r_i]$ ，合并所有有交集的区间（端点相交也算），输出合并后的区间个数。

输入格式

第一行一个整数 n ($1 \leq n \leq 100000$)。

接下来 n 行，每行两个整数 l_i 和 r_i 。

输出格式

一行，合并后的区间个数。

输入样例

```
5
1 2
2 4
5 6
7 8
7 10
```

输出样例

```
3
```

解题思路：

按左端点排序，遍历合并：当前区间左端点 $\leq$ 前一个右端点则合并，否则新开。

复杂度分析：

时间复杂度： $O(n \log n)$ — 排序算法

空间复杂度： $O(1)$

► 参考代码

JD127: 一箭穿心

简单 AcWing 905 | JD127

赵晴儿在数轴上标了 N 个区间，每个区间表示一个敌人的巡逻范围。"用最少的暗器，让每个敌人的巡逻范围里至少中一枚。"

李少白想每个区间放一枚。赵晴儿摇头："贪心——按右端点从小到大排。第一枚暗器射在第一个区间的右端点，这样所有包含这个点的区间都中了。然后跳过这些区间，再射下一批。"

"选右端点是因为它最'靠左'——能覆盖尽可能多的后续区间。"

给定 N 个闭区间 $[a_i, b_i]$ ，选择尽量少的点，使得每个区间内至少有一个点。

输入格式

第一行一个整数 N ($1 \leq N \leq 100000$)。接下来 N 行，每行两个整数 a_i 和 b_i 。

输出格式

一行，最少需要的点数。

输入样例

```
3
1 3
2 4
3 5
```

输出样例

```
1
```

解题思路：

按右端点排序，每次选当前区间的右端点，跳过所有包含该点的区间。

复杂度分析:

时间复杂度: $O(n \log n)$ — 排序算法

空间复杂度: $O(1)$

► [参考代码](#)

JD128: 合果成堆

简单 AcWing 148 | JD128

果园里有 N 堆果子，每堆重量不同。赵晴儿要把它们合并成一堆，每次只能合并两堆，消耗的能量等于两堆重量之和。

"如果先合并重的，后面每次合并都要带着这堆重的走，代价越来越大。"赵晴儿说，"所以反过来——每次选最轻的两堆合并，这样每一步的代价都最小。"

李少白想到用小根堆："每次取出堆顶（最轻的两堆），合并后放回去。总代价最小。"

这就是哈夫曼树的思想。

有 N 堆果子，每次合并两堆（代价为两堆之和），求合并为一堆的最小总代价。

输入格式

第一行一个整数 N ($1 \leq N \leq 10000$)。第二行 N 个整数，表示每堆的重量。

输出格式

一行，最小总代价。

输入样例

```
3
1 2 9
```

输出样例

```
15
```

解题思路：

小根堆：每次取最小的两个合并，结果放回堆中。 $O(n \log n)$ 。

复杂度分析:

时间复杂度: $O(n \log n)$ — 堆操作, 每次插入/删除 $O(\log n)$

空间复杂度: $O(n)$

► [参考代码](#)

JD129: 中位之选

简单 AcWing 104 | JD129

数轴上有 N 个商铺，赵晴儿要建一个货仓，让所有商铺到货仓的货物运输距离之和最小。

"选在哪里?"李少白问。

"排序后取中位数。"赵晴儿说，"数学可以证明：中位数使绝对偏差之和最小。如果选在中位数左边，右边的商铺距离全部变远；选在中位数右边，左边的商铺距离全部变远。只有中位数是平衡点。"

给定 N 个点的坐标，求一个位置使所有点到该位置的绝对距离之和最小，输出最小距离之和。

输入格式

第一行一个整数 N ($1 \leq N \leq 100000$)。第二行 N 个整数，表示各点坐标。

输出格式

一行，最小距离之和。

输入样例

```
5
1 2 3 4 5
```

输出样例

```
6
```

解题思路：

排序后取中位数（下标 $n/2$ ），计算所有点到中位数的距离之和。

复杂度分析:

时间复杂度: $O(n \log n)$ — 排序算法

空间复杂度: $O(1)$

► [参考代码](#)

第12章 利器出鞘

进阶数据结构

兵器阁深处，梁嘉峰打开一个布满机关的箱子："单调栈、单调队列、KMP、堆、并查集——每一种都是利器。"

本章学习进阶数据结构——它们是解决复杂问题的利器，掌握它们，你将能应对更高级的算法挑战。

JD130：铁链串珠

简单 AcWing 826 | JD130

兵器阁深处，尘封的铁架上挂着一条乌黑的铁链。梁嘉峰伸手取下，铁链在灯笼的映照下泛着幽幽冷光，链上串着十余颗拇指大的铁珠，每一颗都刻着不同的符号。"你看，"他指着珠子之间细细的铁丝，"每颗珠子存着一个值，每颗珠子的尾部还连着一根铁丝，指向下一颗——这就是链表。"

赵晴儿接过铁链，手指轻轻拨动珠子，发出清脆的碰撞声。"用数组模拟这条链：head指向第一颗珠子，val[]存每颗珠子的值，ne[]存下一颗珠子的下标。"她边说边比划，"插入、删除都只需要改两根铁丝的指向， $O(1)$ 。"

李少白伸手想从中间拔出一颗珠子，却发现只需解开两根铁丝，前后珠子便自动相连。"链表的好处是插入删除快，"梁嘉峰说，"代价是不能像数组那样随机访问——你必须从头顺着铁丝一颗颗走。"

实现一个单链表，支持在头部插入、在第k个插入的元素后面插入、删除第k个插入的元素前面的元素。

输入格式

若干行操作命令： $H\ x$ （头部插入 x ）、 $I\ k\ x$ （在第 k 个插入的元素后插入 x ）、 $D\ k$ （删除第 k 个插入的元素前面的元素）。

输出格式

输出最终链表，从头到尾空格隔开。

输入样例

```
10
H 9
I 1 1
D 1
H 6
I 3 6
I 4 7
D 3
D 4
I 2 2
```

D 2

输出样例

6

7

解题思路：

数组模拟链表： $head, val[], ne[]$ 。插入：新节点的 ne 指向原 $next$ ，前驱的 ne 指向新节点。

复杂度分析：

时间复杂度： $O(n)$ — 线性遍历处理

空间复杂度： $O(n)$

► [参考代码](#)

JD131: 叠石成塔

简单 AcWing 828 | JD131

兵器阁的墙角，梁嘉峰蹲下身，从地上捡起一块刻着"5"的青石板，稳稳地放在最底层。石板落在地面上，发出一声沉闷的"咚"。他又放上一块刻着"6"的，再放上"3"……一摞石块渐渐成形。"栈——后进先出。只能从顶部放石块，也只能从顶部取。"

赵晴儿数着石块，指尖轻点最顶上那块："push是放石块到顶部，pop是取走顶部的石块，query是看看顶部那块刻着什么数。"

李少白试着从中间抽出一块，整摞石块摇摇欲坠。梁嘉峰按住他的手，笑道："不行——栈只能操作栈顶。你想想，叠盘子、摞书、函数调用……生活中到处是栈。"

实现一个栈，支持push、pop、query操作。

输入格式

若干行操作命令：push x（入栈）、pop（出栈）、query（查询栈顶）。

输出格式

对每个query和pop操作输出一行结果。

输入样例

```
10
push 5
query
push 6
pop
query
pop
push 3
query
push 4
query
```

输出样例

5
5
3
4

解题思路：

数组模拟栈： tt 指向栈顶。 $push: st[++tt]=x$ 。 $pop: tt--$ 。 $query: st[tt]$ 。

复杂度分析：

时间复杂度： $O(n)$ — 栈操作，每个元素最多入栈出栈各一次

空间复杂度： $O(n)$

► 参考代码

JD132：列阵待命

简单 AcWing 829 | JD132

兵器阁外的空地上，晨光斜照。一排新入门的弟子正列队等候分配兵器，从左到右依次站好，脚步声渐次停歇。赵晴儿指着队伍："队列——先进先出。新来的站队尾，走的从队头走。"

梁嘉峰站在队头，点到名字的弟子便出列领剑。"push是从队尾加入，pop是从队头弹出，query是看队头是谁。"他一边说，一边拍拍队头弟子的肩膀。

李少白看着队列缓缓缩短，忽然悟了："排队买饭、消息队列、BFS——原来都是队列！先进先出，天经地义。"

实现一个队列，支持push、pop、query操作。

输入格式

若干行操作命令：push x（入队）、pop（出队）、query（查询队头）。

输出格式

对每个query和pop操作输出一行结果。

输入样例

```
10
push 5
query
push 3
pop
query
push 7
push 8
pop
query
pop
```

输出样例

```
5
```

3
7
7

解题思路：

数组模拟队列：hh队头，tt队尾。push: $q[++tt]=x$ 。pop: $hh++$ 。query: $q[hh]$ 。

复杂度分析：

时间复杂度： $O(n)$ — 队列操作

空间复杂度： $O(n)$

► 参考代码

JD133: 算筹求值

简单 AcWing 3302 | JD133

赵晴儿递来一个算式: $3 + 4 * 2 - (1 + 5)$ 。算出结果。"

梁嘉峰拿出两个盒子: "数字盒和符号盒。遇到数字放数字盒。遇到符号——如果比栈顶符号优先级低,就把栈顶符号和两个数字算掉。遇到左括号直接放,遇到右括号一直算到左括号。"

"用两个栈——数字栈和运算符栈,按优先级处理。"

给定一个包含 $+$ 、 $-$ 、 $*$ 、 $/$ 、 $($ 、 $)$ 的表达式,求值。

输入格式

一行包含 $+$ 、 $-$ 、 $*$ 、 $/$ 、 $($ 、 $)$ 和整数的表达式。

输出格式

一行,表达式的值。

输入样例

$2+3*4$

输出样例

14

解题思路:

双栈法: 数字栈+运算符栈。比较优先级决定是否先算栈顶。

复杂度分析:

时间复杂度: $O(n)$ — 栈操作, 每个元素最多入栈出栈各一次

空间复杂度: $O(n)$

▶ 参考代码

JD134：一山更比

简单 AcWing 830 | JD134

梁嘉峰指着一排山峰："每座山往左看，找到第一座比自己高的山。"

赵晴儿说："用一个栈维护'还没找到更高山'的山峰——从左往右扫，遇到更高的山，栈里比它矮的全部弹出，它们的答案就是这座山。"

"栈里永远是单调递减的——所以叫单调栈。"

给定一个整数序列，对每个元素找到它左边第一个比它大的数。

输入格式

第一行一个整数 n 。第二行 n 个整数。

输出格式

一行，每个元素左边第一个比它大的数，不存在输出-1。

输入样例

```
5
3 2 5 1 4
```

输出样例

```
-1 3 -1 5 5
```

解题思路：

单调栈：遍历时弹出比当前小的元素，栈顶就是答案。

复杂度分析：

时间复杂度： $O(n)$ — 单调栈，每个元素最多入栈出栈各一次

空间复杂度: $O(n)$

▶ [参考代码](#)

JD135: 窗移镜照

简单 AcWing 154 | JD135

赵晴儿在沙盘上画了一排数字，用一个框框住其中 k 个："这个框每往右移一格，告诉我框里的最小值。"

梁嘉峰说："用单调队列——维护一个单调递增的队列。窗口移动时，队头如果过期就弹出；新元素入队时，把队尾比它大的全部弹出。队头永远是当前窗口的最小值。"

"一扇窗户移过整排数字，镜中始终映出最小值。"

给定一个长度为 n 的整数序列和窗口大小 k ，求每个窗口内的最小值。

输入格式

第一行两个整数 n 和 k （窗口大小）。第二行 n 个整数。

输出格式

每个窗口的最小值，空格隔开。

输入样例

```
8 3
1 3 -1 -3 5 3 6 7
```

输出样例

```
-1 -3 -3 -3 3 3
```

解题思路：

单调队列：维护递增队列。窗口滑动时弹出过期元素和比新元素大的元素。

复杂度分析:

时间复杂度: $O(n)$ — 单调队列, 每个元素最多入队出队各一次

空间复杂度: $O(n)$

► [参考代码](#)

JD136: 剑谱寻踪

简单 AcWing 831 | JD136

赵晴儿拿着一本剑谱："在这本厚厚的剑谱里找一段特定的剑招序列——模式串在文本串中出现了几次？ 分别在哪里？ "

李少白想一个个比对，但剑谱有十万字。梁嘉峰说："KMP算法——先预处理模式串，算出每个位置的next数组（最长相等前后缀）。匹配时利用next跳转，不用回退文本指针。 $O(n+m)$ 。"

在文本串中查找模式串的所有出现位置。

输入格式

第一行一个整数 n （模式串长度）。第二行模式串。第三行一个整数 m （文本串长度）。第四行文本串。

输出格式

一行，模式串在文本串中所有出现的起始位置（从0开始），空格隔开。

输入样例

```
3
aba
5
ababa
```

输出样例

```
0 2
```

解题思路：

KMP：预处理next数组（最长相等前后缀），匹配时利用next跳转。

复杂度分析:

时间复杂度: $O(n + m)$ — KMP字符串匹配

空间复杂度: $O(m)$

► [参考代码](#)

JD137: 堆石成丘

简单 AcWing 839 | JD137

兵器阁后院的沙地上，梁嘉峰用石子摆出一棵倒三角形的树。他把最小的石子放在最顶端，稍大的放第二层，更大的放第三层。"堆——一种特殊的完全二叉树。父节点的值永远比子节点小，这叫小根堆。"

赵晴儿蹲下身，手指在沙地上画出编号："用数组存——父节点 i 的子节点是 $2i$ 和 $2i+1$ 。"她捡起一颗新石子，比了比位置，"插入时从底部上浮，删除堆顶时从顶部下沉。"

李少白看着堆顶那颗最小的石子，忽然明白了："所以取最小值只要看堆顶—— $O(1)$ ！"

实现一个小根堆，支持插入、删除第 k 个插入的元素、查询最小值。

输入格式

若干行操作命令： $I\ x$ （插入 x ）、 PM （查询最小值）、 $D\ k$ （删除第 k 个插入的元素）。

输出格式

对每个 PM 操作输出一行最小值。

输入样例

```
5
I 3
I 1
I 4
PM
D 1
```

输出样例

```
1
```

解题思路：

数组模拟堆：上浮和下沉操作。父节点 i 的子节点是 $2i$ 和 $2i+1$ 。

复杂度分析：

时间复杂度： $O(n \log n)$ — 堆操作，每次插入/删除 $O(\log n)$

空间复杂度： $O(n)$

► 参考代码

JD138：合帮并派

简单 AcWing 836 | JD138

江湖上有 N 个门派，每个弟子属于一个门派。赵晴儿问：“两个弟子是不是同门？”梁嘉峰说：“用并查集——合并两个门派用`union`，查询两个人是否同门用`find`。路径压缩后几乎 $O(1)$ 。”

给定 N 个元素和 M 个操作：合并两个集合，或查询两个元素是否在同一集合。

输入格式

第一行两个整数 N 和 M 。接下来 M 行，每行 $M 1 a b$ （合并 a 和 b 所在集合）或 $Q a b$ （查询 a 和 b 是否同属一个集合）。

输出格式

对每个 Q 操作输出一行`Yes`或`No`。

输入样例

```
4 5
M 1 2
Q 1 2
M 2 3
Q 1 3
Q 1 4
```

输出样例

```
Yes
Yes
No
```

解题思路：

并查集：`find`时路径压缩，`union`时合并两棵树。

复杂度分析:

时间复杂度: $O(\alpha(n))$ — 并查集操作, 近似常数时间

空间复杂度: $O(n)$

► [参考代码](#)

JD139: 门派计数

简单 AcWing 837 | JD139

兵器阁的大厅里，赵晴儿铺开一张江湖地图，上面用朱砂标注着数十个门派的位置。她指着地图上两个相邻的门派："上次合并后，华山派现在有多少人？"梁嘉峰一时答不上来。

"这就是问题所在，"赵晴儿说，"光知道谁跟谁同门还不够，还得知道每个门派有多少人。"

梁嘉峰沉思片刻，在沙盘上画出并查集的结构："维护一个`size[]`数组——合并时，把一个门派的人数加到另一个上。查询时直接读`size[find(x)]`， $O(1)$ 。"

李少白看着沙盘上逐渐壮大的门派标记，忽然明白了："原来并查集不仅能判断同门，还能统计人数。"

给定 N 个元素和 M 个操作：合并两个集合，或查询某个元素所在集合的大小。

输入格式

第一行两个整数 N 和 M 。接下来 M 行，每行 $M \ a \ b$ （合并）或 $C \ a$ （查询 a 所在集合大小）。

输出格式

对每个 C 操作输出一行集合大小。

输入样例

```
4 5
M 1 2
C 1
M 2 3
C 1
C 4
```

输出样例

```
2
3
1
```


★
解题思路：

维护size[]数组，合并时更新size。

复杂度分析：

时间复杂度： $O(n)$ — 线性遍历处理

空间复杂度： $O(n)$

► **参考代码**

JD140：三界相克

简单 AcWing 240 | JD140

梁嘉峰说："A吃B，B吃C，C吃A——三种动物形成环形食物链。给你N个动物和K句话，判断哪些话是假的。"

赵晴儿解释："扩展并查集——每个动物拆成三个节点，分别表示'A是A'、'A是B的食物'、'A是C的食物'。用模3的关系判断真假。"

给定N个动物和K句话，判断假话数量。

输入格式

第一行两个整数N和K。接下来K行，每行两个整数x y z ($x=1$ 表示y和z同类， $x=2$ 表示y吃z)。

输出格式

一行，假话的数量。

输入样例

```
100 7
1 101 1
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5
```

输出样例

```
3
```

解题思路：

扩展并查集：每个点拆成3个，模3判断关系。

复杂度分析：

时间复杂度： $O(\alpha(n))$ — 并查集操作，近似常数时间

空间复杂度： $O(n)$

► **参考代码**

第13章 迷雾寻踪

搜索与回溯

迷雾弥漫的林间小径上，赵晴儿指着分叉的路口："DFS 深入探索每一条路，BFS 逐层推进找最短。回溯——选了不行就回头。"

本章学习搜索和回溯——DFS 和 BFS 是图搜索的基础，回溯是枚举所有可能的利器。

JD141: 雾中排阵

简单 AcWing 842 | JD141

迷雾弥漫的林间小径上，梁嘉峰递给李少白 n 枚令牌："按字典序列出1到 n 的所有排列。每步选一个没用过的令牌，选完所有就回头换一个——这就是DFS回溯。"

给定整数 n ，按字典序输出1到 n 的所有全排列。

输入格式

一个整数 n ($1 \leq n \leq 9$)。

输出格式

每行一个排列，数字之间空格隔开，按字典序输出。

输入样例

3

输出样例

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

解题思路：

DFS+回溯：选→递归→撤销。用`used[]`标记。

复杂度分析：

时间复杂度： $O(V + E)$ — DFS遍历，每个节点和边访问一次

空间复杂度: $O(V)$

▶ [参考代码](#)

JD142: 棋盘布后

简单 AcWing 843 | JD142

迷雾林的深处，一张石桌上摆着一方 $n \times n$ 的棋盘，棋子散落一旁。赵晴儿拿起一枚皇后棋子，冰凉的玉石在指尖转动："放 n 个皇后——不能同行、同列、同对角线。有多少种放法？"

李少白将第一枚皇后放在第一行，棋子落下，发出清脆的"嗒"声。他伸手去放第二枚，却发现与第一枚冲突，只好拿起重放。"每行选一个安全的位置放皇后，递归下一行。冲突就回退——这就是回溯。"

梁嘉峰在旁指点："用三个布尔数组标记列和两条对角线， $O(1)$ 判断冲突。选了不行就回头，不要怕试错。"

给定 n ，求 n 皇后的所有解。

输入格式

一个整数 n 。

输出格式

所有解，每个解 n 行（Q表示皇后，.表示空），解之间空行。

输入样例

4

输出样例

```
.Q..
...Q
Q...
..Q.

..Q.
Q...
...Q
```

.Q..

解题思路：

DFS+回溯：逐行放置，标记列和两条对角线。

复杂度分析：

时间复杂度： $O(V + E)$ — DFS遍历，每个节点和边访问一次

空间复杂度： $O(V)$

► 参考代码

JD143: 迷宫寻路

简单 AcWing 844 | JD143

迷雾林的尽头，一道石墙围成的迷宫赫然出现。入口在左上角，一缕微光从石缝中透入；出口在右下角，隐约能听到外面的风声。赵晴儿摸着冰冷的石墙："最短路径有多长？"

梁嘉峰蹲下身，在沙地上画出迷宫的缩略图："BFS——从入口开始，一层一层向外扩展，像水波纹一样。第一次到达出口时，层数就是最短路径。"

李少白闭上眼睛，想象自己站在迷宫入口，每一步都只能走上下左右四个方向。他睁开眼："所以BFS天然适合求最短路——因为它是逐层搜索的。"

给定一个 $n \times m$ 的迷宫（0可走，1是墙），求最短路径长度。

输入格式

第一行两个整数 n 和 m 。接下来 n 行每行 m 个整数（0可走，1是墙）。

输出格式

一行，最短路径长度。不可达输出-1。

输入样例

```
5 5
0 1 0 0 0
0 1 0 1 0
0 0 0 0 0
0 1 1 1 0
0 0 0 1 0
```

输出样例

8

解题思路：

BFS层序遍历，第一次到达终点就是最短路径。

复杂度分析：

时间复杂度： $O(V + E)$ — BFS遍历，每个节点和边访问一次

空间复杂度： $O(V)$

► 参考代码

JD144: 八码迷局

简单 AcWing 845 | JD144

迷雾中出现一个 3×3 数字拼盘，有一个空格。赵晴儿说："从初始状态移到目标状态，最少几步？每次只能把空格和相邻数字交换。"

梁嘉峰说："BFS+状态压缩——把整个拼盘编码成一个字符串，每次移动生成新状态，用哈希表判重。"

给定初始状态和目标状态，求最少移动步数。

输入格式

两行，每行9个字符（ 3×3 拼盘，x表示空格）。第一行初始状态，第二行目标状态。

输出格式

一行，最少移动步数。无解输出-1。

输入样例

```
1 2 3
x 4 6
7 5 8
```

```
1 2 3
4 5 6
7 8 x
```

输出样例

```
2
```

解题思路：

BFS+状态编码为字符串，哈希表判重。

复杂度分析:

时间复杂度: $O(V + E)$ — BFS遍历, 每个节点和边访问一次

空间复杂度: $O(V)$

► 参考代码

JD145：古树重心

简单 AcWing 846 | JD145

迷踪林深处，一棵千年古树矗立在薄雾中。树干粗壮如磨盘，枝桠向四面八方伸展，遮天蔽日。梁嘉峰仰头望着繁茂的树冠："找到它的重心——删除后最大子树最小的那个节点。"

赵晴儿绕着古树走了一圈，手指轻抚粗糙的树皮："DFS遍历树，对每个节点计算：删掉它后，最大子树有多少个节点。取最小的那个——那个节点就是重心。"

李少白抬头看着树冠，忽然明白了："重心就像大树的平衡点——砍掉它，剩下的枝桠不会一边太重。"

给定一棵树，求其重心和最大子树的最小大小。

输入格式

第一行一个整数 n 。接下来 $n-1$ 行每行两个整数表示一条边。

输出格式

一行，最大子树的最小大小。

输入样例

```
9
1 2
1 7
1 4
2 8
2 5
4 3
4 6
6 9
```

输出样例

```
4
```

解题思路：

DFS遍历树，计算每个节点的最大子树大小。重心= $\max(n-\text{subSize}, \text{各子树size})$ 最小的节点。

复杂度分析:

时间复杂度: $O(V + E)$ - DFS遍历, 每个节点和边访问一次

空间复杂度: $O(V)$

► [参考代码](#)

JD146：城池层递

简单 AcWing 847 | JD146

迷雾散去，前方出现一片开阔的平原，远处城池星罗棋布。赵晴儿铺开城池图，墨迹尚未干透："城池之间有道路相连，但道路有方向。从城1出发，到每个城池的最短距离是多少？"

梁嘉峰用手指在图上比划："BFS——从城1开始，每层扩展一步。第一次到达的城池，层数就是最短距离。"他顿了顿，"前提是边权都相等——如果边权不同，就得用Dijkstra了。"

李少白看着地图上逐渐被标记的城池，忽然明白了："BFS就像涟漪扩散——离得近的城池先被波及，离得远的后被波及。"

给定一个有向图和起点1，求每个点到起点的最短距离。

输入格式

第一行两个整数 n 和 m 。接下来 m 行每行两个整数 a b 表示一条有向边。

输出格式

一行， n 个整数表示每个点到点1的最短距离，不可达输出-1。

输入样例

```
4 4
1 2
2 3
1 3
3 4
```

输出样例

```
0 1 1 2
```

解题思路：

BFS层序遍历，每层距离+1。

复杂度分析：

时间复杂度： $O(V + E)$ - BFS遍历，每个节点和边访问一次

空间复杂度: $O(V)$

▶ [参考代码](#)

第14章 千里奔袭

最短路与生成树

城际连横，梁嘉峰展开地图："从一座城到另一座城，最短的路在哪里？Dijkstra、Floyd、Kruskal——每一种算法都是战场上的利器。"

本章学习最短路和最小生成树——图论的核心算法，掌握它们，你将能解决任何网络优化问题。

JD147: 拓扑之序

简单 AcWing 848 | JD147

城际连横。赵晴儿铺开城池图："城池之间有方向道路。排出一个合法的建设顺序——每条道路都是从先建的城池指向后建的。"

梁嘉峰说："拓扑排序——找入度为0的点，输出后删除它的出边，再找下一个入度为0的点。如果有环，就不可能全部输出。"

给定一个有向图，输出拓扑序。有环则输出-1。

输入格式

第一行两个整数 n 和 m 。接下来 m 行每行两个整数 a b 表示一条有向边。

输出格式

一行，拓扑序，空格隔开。有环输出-1。

输入样例

```
3 3
1 2
2 3
1 3
```

输出样例

```
1 2 3
```

解题思路：

BFS+入度维护。有环则无解。

复杂度分析：

时间复杂度： $O(V + E)$ — BFS遍历，每个节点和边访问一次

空间复杂度： $O(V)$

▶ 参考代码

JD148：近者先行

简单 AcWing 849 | JD148

赵晴儿在城池图上标了距离："从城1出发，到每个城池的最短距离是多少？边权都是正数。"

梁嘉峰说："朴素Dijkstra—每次选未访问的最近点，用它更新邻居的距离。选过的点不再更新。"

给定一个带权有向图和起点，求到每个点的最短距离。

输入格式

第一行两个整数 n 和 m 。接下来 m 行每行三个整数 a b w 表示一条从 a 到 b 的权为 w 的有向边。

输出格式

一行， n 个整数表示到点1的最短距离。

输入样例

```
3 3
1 2 2
2 3 1
1 3 4
```

输出样例

```
0 2 3
```

解题思路：

Dijkstra: $O(n^2)$ 。每次选最近未访问点，更新邻居。

复杂度分析：

时间复杂度: $O((V + E) \log V)$ — Dijkstra最短路，优先队列优化

空间复杂度: $O(V + E)$

► 参考代码

JD149: 堆中取近

简单 AcWing 850 | JD149

梁嘉峰指着地图上密密麻麻的城池，却只有稀疏的几条道路相连："点很多，边很少——朴素Dijkstra每次遍历所有点找最近的，太慢了。"

赵晴儿眼睛一亮："用小根堆代替遍历——每次从堆顶取最近的未访问点， $O(\log n)$ 。"她拿起一把石子，在沙地上摆出堆的结构，"堆优化后，总复杂度从 $O(n^2)$ 降到 $O(m \log n)$ ——边少的时候快得多。"

李小白看着堆顶那颗最小的石子，忽然明白了："原来堆就是Dijkstra的加速器！"

给定一个带权有向图和起点，求到每个点的最短距离。边数远小于点数的平方。

输入格式

第一行两个整数 n 和 m 。接下来 m 行每行三个整数 a b w 。

输出格式

一行，到点1的最短距离。

输入样例

```
3 3
1 2 2
2 3 1
1 3 4
```

输出样例

```
0 2 3
```

解题思路：

堆优化Dijkstra: $O(m \log n)$ 。用小根堆取最近点。

复杂度分析：

时间复杂度: $O((V + E) \log V)$ — Dijkstra最短路，优先队列优化

空间复杂度: $O(V + E)$

▶ 参考代码

JD150：队中松弛

简单 AcWing 851 | JD150

赵晴儿指着图上的负权边："Dijkstra处理不了负权边。用SPFA—BFS+松弛。把待更新的点放入队列，每次取一个点，用它更新邻居。邻居变短了就入队。"

给定一个可能有负权边的图，求最短距离。

输入格式

第一行两个整数 n 和 m 。接下来 m 行每行三个整数 a b w (w 可为负)。

输出格式

一行，到点1的最短距离。

输入样例

```
3 3
1 2 -1
2 3 2
1 3 3
```

输出样例

```
0 -1 1
```

解题思路：

SPFA：队列优化Bellman-Ford。

复杂度分析：

时间复杂度： $O(k \times E)$ — SPFA最短路， k 为平均入队次数

空间复杂度： $O(V + E)$

► 参考代码

JD151: 万径归一

简单 AcWing 854 | JD151

梁嘉峰铺开全图，一张巨大的城池网络图铺满了整张桌子。墨线纵横交错，连接着每一座城池。"多源最短路——任意两点之间的最短距离，都要算出来。"

赵晴儿皱眉："那要跑 n 次Dijkstra？"梁嘉峰摇头，拿起毛笔在图边写下三行代码："Floyd算法——三重循环。外层枚举中间点 k ，内层枚举起点 i 和终点 j 。 $dp[i][j] = \min(dp[i][j], dp[i][k] + dp[k][j])$ 。"

李小白盯着公式看了许久，忽然悟了："就像在图上架桥——如果经过 k 点能缩短 i 到 j 的距离，就更新。三层循环下来，所有最短路都算好了。"

给定一个带权图，求任意两点之间的最短距离。

输入格式

第一行三个整数 n 、 m 和 q 。接下来 m 行每行三个整数 a b w 。接下来 q 行每行两个整数 a b 。

输出格式

q 行，每行一个整数。不可达输出impossible。

输入样例

```
3 3 2
1 2 2
2 3 1
1 3 4
1 3
2 1
```

输出样例

```
3
3
```

解题思路：

Floyd: $O(n^3)$ 。三重循环枚举中间点。

复杂度分析:

时间复杂度: $O(V^3)$ - Floyd全源最短路, 三重循环

空间复杂度: $O(V^2)$

► [参考代码](#)

JD152: 蔓延成网

简单 AcWing 858 | JD152

赵晴儿指着散落的城池："用最少的路把所有城池连起来，总路长最短。"

梁嘉峰说："Prim算法—类似Dijkstra，每次选离已连通部分最近的未加入点，把那条边加入生成树。"

给定一个带权无向图，求最小生成树的总权值。

输入格式

第一行两个整数 n 和 m 。接下来 m 行每行三个整数 a b w 。

输出格式

一行，最小生成树的总权值。不连通输出impossible。

输入样例

```
4 5
1 2 1
1 3 2
2 3 3
2 4 4
3 4 5
```

输出样例

```
7
```

解题思路：

Prim: $O(n^2)$ 。每次选最近未加入点。

复杂度分析：

时间复杂度: $O(E \log E)$ — 最小生成树，排序边后并查集

空间复杂度: $O(V + E)$

► 参考代码

JD153: 逐边成林

简单 AcWing 859 | JD153

散落的城池之间，道路纵横。梁嘉峰把所有道路写在竹简上，按造价从低到高排好："Kruskal——按边权从小到大排序，逐条加入。"

他拿起最便宜的那条路，看了看两端的城池，用并查集查了查："这两座城已经在同一个连通块里了？跳过。"他又拿起下一条："不在同一个连通块？加入！"

赵晴儿看着逐渐成形的路网，忽然明白了："并查集在这里的作用就是判环——如果两个端点已经连通，再加边就会形成环，那就不是树了。"

给定一个带权无向图，求最小生成树的总权值。

输入格式

第一行两个整数 n 和 m 。接下来 m 行每行三个整数 a b w 。

输出格式

一行，最小生成树的总权值。不连通输出impossible。

输入样例

```
4 5
1 2 1
1 3 2
2 3 3
2 4 4
3 4 5
```

输出样例

```
7
```

解题思路：

Kruskal： $O(m\log m)$ 。按边权排序，并查集判环。

复杂度分析:

时间复杂度: $O(E \log E)$ - 最小生成树, 排序边后并查集

空间复杂度: $O(V + E)$

► [参考代码](#)

JD154：二色分界

简单 AcWing 860 | JD154

赵晴儿指着沙盘上的一个图，节点之间连线纵横交错："能不能把所有点分成两组，同组之间没有直接相连的边？"

梁嘉峰拿起红蓝两色的石子："染色法——从任意点开始，染红色；它的邻居染蓝色；邻居的邻居染红色……"他一边说，一边在沙盘上摆放石子。忽然，他停住了一个节点的邻居既需要红色又需要蓝色。"冲突了——不是二分图。"

李小白看着沙盘上红蓝相间的石子，忽然明白了："BFS或DFS染色，冲突就返回`false`。这就是二分图判定的本质。"

给定一个无向图，判断是否是二分图。

输入格式

第一行两个整数 n 和 m 。接下来 m 行每行两个整数 a b 。

输出格式

一行，`Yes`或`No`。

输入样例

```
4 4
1 2
2 3
3 4
4 1
```

输出样例

```
Yes
```

解题思路：

BFS/DFS染色，冲突则不是二分图。

复杂度分析:

时间复杂度: $O(V + E)$ - BFS遍历, 每个节点和边访问一次

空间复杂度: $O(V)$

► [参考代码](#)

第15章 华山论剑

动态规划

华山之巅，风雷激荡。四面八方的高手齐聚于此——华山论剑，一决高下。"动态规划——记住过去，优化未来。背包、区间、状态压缩、树形——每一种DP都是一门绝技。"

本章学习动态规划——算法世界上最精妙的技巧。掌握DP，你将拥有解决任何复杂问题的能力。这一战之后，你便可以出师了。

JD155：一剑一鞘

简单 AcWing 2 | JD155

华山脚下，试剑台前。西域刀客赫连铁拔出背上唯一的刀鞘，鞘中却藏着 N 柄短刀，每柄有重量和锋芒。刀鞘容量有限——选哪些刀，才能让锋芒之和最大？

李小白拔剑上前。梁嘉峰低声道："每柄刀只能选一次——01背包。逆序遍历容量， $dp[j] = \max(dp[j], dp[j-v]+w)$ 。"

李小白深吸一口气，提笔写下。赫连铁看了答案，缓缓点头："第一关，过了。"

输入格式

第一行两个整数 N 和 V 。接下来 N 行每行两个整数 v 和 w 。

输出格式

一行，最大价值。

输入样例

```
4 5
1 2
2 4
3 4
4 5
```

输出样例

```
8
```

解题思路：

01背包： $dp[j] = \max(dp[j], dp[j-v[i]]+w[i])$ ，逆序遍历 j 。

复杂度分析：

时间复杂度： $O(n \times v)$ — 01背包DP，逆序遍历容量

空间复杂度： $O(v)$

▶ 参考代码



JD156：无穷剑阵

简单 AcWing 3 | JD156

第二阵。铸剑山庄庄主出场，身后跟着仆从，抬着无数柄相同的剑。"每柄剑重 v ，锋利 w 。我的剑取之不尽——你能装多少？"

赵晴儿上前一步："完全背包——每种剑可以拿无限柄，内层循环正序遍历。"她提笔如飞，片刻交卷。庄主愕然："这么快？"

输入格式

第一行两个整数 N 和 V 。接下来 N 行每行两个整数 v 和 w 。

输出格式

一行，最大价值。

输入样例

```
4 5
1 2
2 4
3 4
4 5
```

输出样例

```
10
```

解题思路：

完全背包：内层正序遍历。每种物品可选无限次。

复杂度分析：

时间复杂度： $O(n \times v)$ — 完全背包DP，正序遍历容量

空间复杂度： $O(v)$

► 参考代码

JD157: 三角登峰

简单 AcWing 898 | JD157

华山云梯，台阶排成三角形，每级刻着一个数字。从顶到底，每步只能向左下或右下。"找出一条路径，使数字之和最大。"裁判宣布。

李小白仰望云梯，忽然笑了："从底向上——每级取下方两个方向中较大的，加上自己。一层层往上推，山顶就是答案。"

输入格式

第一行一个整数 n 。接下来 n 行第 i 行有 i 个整数。

输出格式

一行，最大路径和。

输入样例

```
5
7
3 8
8 1 0
2 7 4 4
4 5 2 6 5
```

输出样例

```
30
```

解题思路：

从底向上DP。 $dp[i][j] = \max(dp[i+1][j], dp[i+1][j+1]) + a[i][j]$ 。

复杂度分析：

时间复杂度： $O(n \times m)$ — 二维DP状态转移

空间复杂度： $O(n \times m)$

► 参考代码

JD158：递增剑意

简单 AcWing 895 | JD158

擂台上，青城派弟子摆出一排剑，每把刻着一个数字。"选出最长的递增序列—数字必须递增，不必相邻。"

李小白用 $O(n^2)$ 的DP接下第一招。梁嘉峰却摇头："后面数据更大。贪心+二分—维护最小末尾数组， $O(n\log n)$ 。"

李小白重写代码，青城弟子面色一变："好快的剑！"

输入格式

第一行一个整数 n 。第二行 n 个整数。

输出格式

一行，最长递增子序列的长度。

输入样例

```
7
3 1 2 1 8 5 6
```

输出样例

```
4
```

解题思路：

贪心+二分：维护最小末尾数组 $q[]$ ，二分查找替换。 $O(n\log n)$ 。

复杂度分析：

时间复杂度： $O(n \log n)$ — 贪心+二分维护最小末尾数组

空间复杂度： $O(n)$

► 参考代码

JD159：双谱共鸣

简单 AcWing 897 | JD159

峨眉派女侠递来两卷剑谱："找出两谱中最长的公共剑招序列——顺序一致但不必连续。"

赵晴儿接过剑谱："LCS—— $dp[i][j]$ 表示前 i 招和前 j 招的最长公共子序列。相等时加一，否则取较大值。"

女侠看了答案，微微一笑："峨眉输了。"

输入格式

第一行两个整数 n 和 m 。第二行长度为 n 的字符串。第三行长度为 m 的字符串。

输出格式

一行，最长公共子序列的长度。

输入样例

```
4 5
abcd
acebd
```

输出样例

```
3
```

解题思路：

LCS：二维DP。相等时 $dp[i][j]=dp[i-1][j-1]+1$ ，否则取 $\max(dp[i-1][j], dp[i][j-1])$ 。

复杂度分析：

时间复杂度： $O(n \times m)$ — 二维DP求最长公共子序列

空间复杂度： $O(n \times m)$

► 参考代码

JD160：合石成堆

简单 AcWing 282 | JD160

华山脚下的石阵— N 堆石子排成一行，每次只能合并相邻两堆，代价是两堆之和。"全部合并的最小代价？"

少林高僧双手合十："贪心不行，必须区间DP。枚举分割点 k ， $dp[i][j] = \min(dp[i][k] + dp[k+1][j]) + \text{sum}(i..j)$ 。"

李少白闭目凝神，三重循环缓缓展开。高僧点头："施主悟了。"

输入格式

第一行一个整数 N 。第二行 N 个整数。

输出格式

一行，最小总代价。

输入样例

```
4
1 3 5 2
```

输出样例

```
22
```

解题思路：

区间DP：枚举分割点。 $dp[i][j] = \min(dp[i][k] + dp[k+1][j]) + \text{sum}(i..j)$ 。 $O(n^3)$ 。

复杂度分析：

时间复杂度： $O(n^3)$ — 区间DP，三重循环枚举区间和分割点

空间复杂度： $O(n^2)$

► 参考代码

JD161: 改字成经

简单 AcWing 902 | JD161

武当道长递来两卷经文: "把A卷改成B卷, 最少几步? 增、删、改, 每次算一步。"

李小白沉思: " $dp[i][j]$ 表示A的前*i*个改成B的前*j*个的最少操作。三种操作取 $\min+1$ 。"

道长抚须而笑: "小友的剑法, 已经入了化境。"

输入格式

第一行两个整数*n*和*m*。第二行长度为*n*的字符串A。第三行长度为*m*的字符串B。

输出格式

一行, 编辑距离。

输入样例

```
3 3
abc
adc
```

输出样例

```
1
```

解题思路:

编辑距离: $dp[i][j] = \min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+cost)$ 。

复杂度分析:

时间复杂度: $O(n \times m)$ — 编辑距离DP, 二维状态转移

空间复杂度: $O(n \times m)$

► 参考代码

JD162：雪道寻长

简单 AcWing 901 | JD162

华山后山，白雪皑皑。一个 $R \times C$ 的滑雪场，从任意高处滑向相邻的低处。"最长能滑多远？"

赵晴儿踏雪而行："记忆化搜索—DFS+缓存。每个格子只算一次，取四个方向中最长的路径加一。"

她的身影在雪道上飞驰，划出一道完美的弧线。

输入格式

第一行两个整数 R 和 C 。接下来 R 行每行 C 个整数表示高度。

输出格式

一行，最长递减路径的长度。

输入样例

```
5 5
1 2 3 4 5
16 17 18 19 6
15 24 25 20 7
14 23 22 21 8
13 12 11 10 9
```

输出样例

```
25
```

解题思路：

记忆化搜索：DFS+缓存。每个格子只算一次，取四个方向中最长路径+1。

复杂度分析：

时间复杂度： $O(n \times m)$ — 记忆化搜索，每个状态只计算一次

空间复杂度： $O(n \times m)$

► 参考代码

JD163：背包叠甲

简单 AcWing 4 | JD163

论剑峰上，铠甲大师出场。他有 N 种铠甲，每种有重量、防御和库存。"每种最多拿 s 件——你能装多少防御？"

李小白眉头一皱："逐件拆分太慢.....二进制优化！把 s 拆成1、2、4、8...的和，转化为01背包。"

铠甲大师大惊："你竟然知道这种方法！"

输入格式

第一行两个整数 N 和 V 。接下来 N 行每行三个整数 v 、 w 和 s （体积、价值、数量）。

输出格式

一行，最大价值。

输入样例

```
3 12
76 84 46
34 31 44
```

输出样例

```
0
```

解题思路：

多重背包二进制优化：拆分 s 为2的幂次，转化为01背包。 $O(N*V*\log S)$ 。

复杂度分析：

时间复杂度： $O(n \times v)$ — 01背包DP，逆序遍历容量

空间复杂度： $O(v)$

► 参考代码

JD164：背包限重

简单 AcWing 5 | JD164

华山论剑第二轮，铠甲大师的弟子出场。他搬出更大的箱子，里面的铠甲种类和数量都翻了数倍。"数据更大了—— N 和 V 都到了极限。"他冷笑着看向李少白，"同样的方法还管用吗？"

台下观众窃窃私语。梁嘉峰却稳坐不动，只传了四个字："二进制优化。"

李少白闭上眼睛，脑海中浮现出二进制拆分的图景——把 s 拆成1、2、4、8……的和，每一份都是2的幂次。他睁开眼，提笔如飞，代码一气呵成。

对手看着李少白提交的答案，脸色骤变："你……你竟然用了二进制优化！"

输入格式

第一行两个整数 N 和 V 。接下来 N 行每行三个整数 v 、 w 和 s 。

输出格式

一行，最大价值。

输入样例

```
680 1562
866 815 599
1355 680 202
```

输出样例

```
742560
```

解题思路：

多重背包二进制优化。与JD163相同思路，数据范围更大。

复杂度分析：

时间复杂度： $O(n \times v \times \log s)$ — 多重背包二进制优化，物品拆分后做01背包

空间复杂度： $O(n \times v)$

▶ 参考代码

JD165：分组选宝

简单 AcWing 9 | JD165

华山论剑的宝物殿中，宝物被分成了若干组，每组用红绸隔开。丐帮长老手持竹棒，指着宝物架："每组只能选一件——选多了，红绸就会断裂，宝物尽毁。"

李小白看着琳琅满目的宝物，沉吟片刻："分组背包——三重循环：外层遍历组，内层逆序容量，最内层遍历组内物品。每组选最优的那一件。"

长老竹棒一点，试探道："小子，你确定每组只选一件？贪心选最贵的不行吗？"

李小白摇头："不行——有些宝物虽然便宜，但轻便，能省下空间装其他组的宝物。必须用DP权衡全局。"长老哈哈大笑，收起竹棒："好！这一关，你过了！"

输入格式

第一行两个整数 N 和 V 。接下来每组：第一行 S （组内物品数），接下来 S 行每行两个整数 v 和 w 。

输出格式

一行，最大价值。

输入样例

```
71 46
80
84 76
```

输出样例

```
1869
```

解题思路：

分组背包：外层遍历组，内层逆序容量，最内层遍历组内物品。

复杂度分析：

时间复杂度： $O(n \times v)$ — 分组背包DP，每组选一个物品

空间复杂度： $O(v)$

► 参考代码

JD166：递增剑意II

简单 AcWing 896 | JD166

擂台第二轮一剑的数量到了十万。 $O(n^2)$ 的剑法已经不够用了。

梁嘉峰在台下传音："贪心+二分——维护最小末尾数组， $O(n\log n)$ 。这是剑法的极致。"

李少白闭上眼睛，剑意如丝，层层递进。对手的剑阵瞬间崩溃。

输入格式

第一行一个整数 N 。第二行 N 个整数。

输出格式

一行，最长递增子序列的长度。

输入样例

```
7
3 1 2 1 8 5 6
```

输出样例

```
4
```

解题思路：

贪心+二分：维护最小末尾数组 $q[]$ ，二分查找替换。 $O(n\log n)$ 。

复杂度分析：

时间复杂度： $O(n \log n)$ — 贪心+二分维护最小末尾数组

空间复杂度： $O(n)$

► 参考代码

JD167：整分剑法

简单 AcWing 900 | JD167

对手是一位数学大师，他写下数字 n ："把它拆成若干正整数之和，有多少种拆法？"

赵晴儿上前："完全背包——从1到 n ，每个数可以用无限次。 $dp[j] += dp[j-i]$ ，模 $1e9+7$ 。"

大师推了推眼镜："你用背包来解整数划分？有意思。"

输入格式

一个整数 n ($1 \leq n \leq 1000$)。

输出格式

一行，划分方案数模 $1e9+7$ 。

输入样例

5

输出样例

7

解题思路：

完全背包DP： $dp[j] += dp[j-i]$ ， i 从1到 n 。模 $1e9+7$ 。

复杂度分析：

时间复杂度： $O(n \times v)$ — 完全背包DP，正序遍历容量

空间复杂度： $O(v)$

► 参考代码

JD168：数码计数

简单 AcWing 338 | JD168

华山石壁上刻着从 a 到 b 的所有数字。"0到9每个数字各出现了几次？"对手出了一道算术题。

李少白没有一个个数。"数位统计——对每个数字和位置，用数学公式算出现次数。 $f(b) - f(a-1)$ 。"

对手目瞪口呆："你.....你怎么算得这么快？"

输入格式

一行两个整数 a 和 b 。

输出格式

一行，10个整数，分别表示数字0~9在 $[a, b]$ 中出现的次数，空格隔开。

输入样例

```
1 10
```

输出样例

```
1 2 1 1 1 1 1 1 1 1
```

解题思路：

数位统计：对每个数字和位置用数学公式计算出现次数。 $f(b) - f(a-1)$ 。

复杂度分析：

时间复杂度： $O(\log n)$ — 数位统计，按位分析贡献

空间复杂度： $O(\log n)$

► 参考代码

JD169：棋盘铺阵

简单 AcWing 291 | JD169

华山之巅，云雾缭绕。四位绝顶高手已经等候多时。

第一位是棋圣。他摆出一个 $n \times m$ 的棋盘和一堆 1×2 的骨牌："铺满整个棋盘，有多少种铺法？"

赵晴儿凝视棋盘，忽然眼中一亮："状压DP——逐列处理。用二进制掩码表示哪些格子被前一列的横骨牌占了。"

棋圣抚掌而笑："好一个状压！老夫输了。"

输入格式

多组数据，每组两个整数 n 和 m 。以0 0结束。

输出格式

每组一行，铺法总数。

输入样例

```
2 3
```

输出样例

```
3
```

解题思路：

状压DP：逐列处理，状态掩码表示被前一列横骨牌占的格子。

复杂度分析：

时间复杂度： $O(2^n \times n)$ — 状压DP，状态压缩枚举子集

空间复杂度： $O(2^n)$

► 参考代码

JD170：万城巡游

简单 AcWing 91 | JD170

第二位绝顶高手——游侠。他铺开一张城池图："从城0出发，每座城恰好走一次，到城 $n-1$ 。最短路径？"

李少白眼中精光一闪："状压DP—— $dp[mask][i]$ 表示访问集合 $mask$ 、当前在城 i 的最小代价。枚举所有状态和转移。 $O(2^n \times n^2)$ 。"

游侠击节赞叹："好！这一剑，我甘拜下风！"

输入格式

第一行一个整数 n 。接下来 n 行每行 n 个整数，表示邻接矩阵。

输出格式

一行，最短Hamilton路径长度。

输入样例

```
5
0 2 4 5 1
2 0 6 5 3
4 6 0 8 3
5 5 8 0 5
1 3 3 5 0
```

输出样例

```
19
```

解题思路：

状压DP： $dp[mask][i]$ 表示访问集合 $mask$ 、当前在城 i 的最小代价。 $O(2^n * n^2)$ 。

复杂度分析：

时间复杂度： $O(2^n \times n^2)$ — 状压DP，状态数 2^n ，每个状态转移 $O(n)$

空间复杂度： $O(2^n \times n)$

► 参考代码

JD171: 群经改字

简单 AcWing 899 | JD171

第三位——藏经阁阁主。他取出一本字典和一段经文："字典里有多少个词，和这段经文的编辑距离不超过 k ？"

赵晴儿沉吟片刻："对字典里每个词，算它和经文的编辑距离。标准编辑距离 DP —— $dp[i][j] = \min(\text{三种操作})$ 。"

阁主长叹："当年我花了十年才悟出此法，你竟片刻之间……"

输入格式

第一行两个整数 N 和 M 。接下来 N 行每行一个字符串（字典）。接下来 M 行每行一个字符串和一个整数 k 。

输出格式

M 行，每行一个整数（字典中编辑距离 $\leq k$ 的字符串个数）。

输入样例

```
3 2
abc
def
1 abc adc
```

输出样例

```
1
```

解题思路：

编辑距离 DP : $dp[i][j] = \min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+cost)$ 。

复杂度分析：

时间复杂度: $O(n \times m)$ — 编辑距离 DP ，二维状态转移

空间复杂度: $O(n \times m)$

► 参考代码

JD172：千机棋局

简单 AcWing 285 | JD172

华山绝顶，风雷激荡。最后一位对手——棋魔，传说中的千机棋圣。他缓缓展开一张巨大的人员图谱："这是宗门的命脉——每个人有一个直属上司。现在要派人出征，但一个士兵和他的上司不能同时出征——否则后方崩溃。每个士兵有不同的战斗力。"

梁嘉峰在台下喊道："树形DP! $dp[u][0]$ 表示 u 留守时子树的最大战力， $dp[u][1]$ 表示 u 出征时。出征则下属必须留守，留守则下属可出可留。找到根节点，DFS一遍！"

李少白深吸一口气，闭上眼睛。他想起了第一天来到剑道宗的那个下午——梁嘉峰递给他两枚铁令，赵晴儿在沙盘上画圆。从变量到递归，从数组到图论，从排序到动态规划.....每一步都是剑道的修行。

他睁开眼睛，提笔写下最后一行代码。

棋魔看着答案，沉默良久，终于起身，深深一揖："华山论剑，到此为止。"

梁嘉峰走上前来，拍了拍李少白的肩膀："恭喜。你可以出师了。"

赵晴儿在一旁笑了："别急，回去还得教下一届呢。"

输入格式

第一行一个整数 N 。第二行 N 个整数（每个员工的快乐值）。接下来 $N-1$ 行每行两个整数 $l\ k$ （ l 的上司是 k ）。

输出格式

一行，最大总快乐值。

输入样例

```
7
1 1 1 1 1 1 1
2 6
3 6
4 6
6 7
5 7
```

输出样例

```
5
```

解题思路：

树形DP: $dp[u][0/1]$ 表示 u 不出征/出征时的最大值。DFS从根遍历。 $dp[u][1] = h[u] + \sum dp[v][0]$,
 $dp[u][0] = \sum \max(dp[v][0], dp[v][1])$ 。

复杂度分析:

时间复杂度: $O(n)$ — 树形DP, DFS遍历每个节点一次

空间复杂度: $O(n)$

► [参考代码](#)

